

# atividade2

June 29, 2026

## 1 Atividade Prática

- **Tema:** tarefa de classificação.
- **Prazo de entrega:** 01 de Junho.

Envie o notebook executado em formato **ipynb** pelo [formulário](#).

---

### 1.1 Enunciado:

Vamos realizar uma tarefa de **análise de sentimentos** com o conjunto de dados [IMDB movie reviews](#), também disponível no diretório dataset/. Realize os seguintes passos para resolver a tarefa de classificação:

#### 1.1.1 1º Passo:

- Importe o *dataset*, embaralhe as amostras, e defina os dados textuais e alvo;
- **Plote** um histograma com os tamanhos das sentenças, a fim de definir um bom tamanho para a entrada do modelo.

#### 1.1.2 Imports, variáveis globais e leitura do dataset

```
[1]: # Não precisamos montar o drive pois os arquivos estão disponíveis localmente
#from google.colab import drive
#drive.mount('/content/drive/')
```

```
[66]: from collections import Counter
from keras import layers, Model
import keras_tuner as kt
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import tensorflow as tf
from matplotlib.ticker import FuncFormatter
import os
from pathlib import Path
from sklearn.metrics import mean_absolute_error, mean_squared_error, \
    median_absolute_error, r2_score
```

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
import tensorboard
from tensorboard.backend.event_processing.event_accumulator import
    EventAccumulator

from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score,
    classification_report, confusion_matrix, f1_score, precision_score,
    recall_score, roc_auc_score

RANDOM_STATE = 42

```

```

[3]: print(tf.config.list_physical_devices())
      print(tf.config.list_physical_devices("GPU"))
      #tf.debugging.set_log_device_placement(True)

```

```

[PhysicalDevice(name='/physical_device:CPU:0', device_type='CPU'),
PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

```

Lemos o dataset local, embaralhamos utilizando `RANDOM_STATE` como semente para garantir reprodutibilidade do experimento, e por fim mostramos o shape e uma pequena amostra dos dados.

```

[4]: df = pd.read_csv('./dataset/imdb.csv', ).sample(frac=1,
    random_state=RANDOM_STATE).reset_index(drop=True)

print(df.shape)
df.head()

```

(50000, 2)

```

[4]:
      reviews  label
0  River's Edge is more than just the story of a ...      1
1  Evening is the beautiful story of the flawed l...      1
2  I question anyone saying they don't care for t...      1
3  This docu-drama is what you would expect from ...      1
4  As several posters have "hinted," this is a so...      0

```

### 1.1.3 Definindo uma função de pré-processamento para o texto

Definimos a função de pré-processamento no início para aplicá-la antes de analisar o tamanho das entradas. Assim operamos sobre um conjunto de dados limpo desde as definições preliminares.

```

[5]: def Preprocessor(x):

      # tudo em minúsculo
      x = tf.strings.lower(x)

      # remove tags html

```

```

x = tf.strings.regex_replace(x, r"<[^>]+>", " ")

# remove caracteres especiais
x = tf.strings.regex_replace(x, r"^\w\s", "")

# remove espaços em excesso
x = tf.strings.regex_replace(x, r" {2,}", " ")

# remove espaços extras no início e no final
x = tf.strings.strip(x)

return x

print (Preprocessor(" Hello, the world's ! ").numpy().decode("utf-8"))

```

hello the worlds

```

2026-05-31 17:55:41.380007: I metal_plugin/src/device/metal_device.cc:1154]
Metal device set to: Apple M2 Max
2026-05-31 17:55:41.380029: I metal_plugin/src/device/metal_device.cc:296]
systemMemory: 32.00 GB
2026-05-31 17:55:41.380035: I metal_plugin/src/device/metal_device.cc:313]
maxCacheSize: 12.48 GB
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
I0000 00:00:1780260941.380047 547783 pluggable_device_factory.cc:305] Could not
identify NUMA node of platform GPU ID 0, defaulting to 0. Your kernel may not
have been built with NUMA support.
I0000 00:00:1780260941.380064 547783 pluggable_device_factory.cc:271] Created
TensorFlow device (/job:localhost/replica:0/task:0/device:GPU:0 with 0 MB
memory) -> physical PluggableDevice (device: 0, name: METAL, pci bus id:
<undefined>)

```

Agora utilizamos a função de processamento para limpar o dataset.

```

[6]: df_clean = df.copy()

df_clean["reviews"] = df_clean["reviews"].apply(
    lambda x: Preprocessor(x).numpy().decode("utf-8")
)

df_clean.head()

```

```

[6]:
      reviews  label
0  rivers edge is more than just the story of a m...      1
1  evening is the beautiful story of the flawed l...      1
2  i question anyone saying they dont care for th...      1
3  this docudrama is what you would expect from r...      1
4  as several posters have hinted this is a sorry...      0

```

Mas definimos X e y em função do dataset original df, pois vamos adicionar a função de pré-processamento no pipeline.

```
[7]: X = df["reviews"]
     y = df["label"]
```

#### 1.1.4 Analisando a limpeza dos textos (função de Preprocessor)

Primeiro uma revisão básica do dataset

```
[8]: print(f"Quantidade de amostras: {len(df_clean)}")

     print("\nValores ausentes:")
     print(df_clean[["reviews", "label"]].isna().sum())

     print("\nReviews vazias após a limpeza:")
     print((df_clean["reviews"].str.len() == 0).sum())

     print("\nDistribuição das classes:")
     print(y.value_counts())
```

Quantidade de amostras: 50000

Valores ausentes:

```
reviews    0
label      0
dtype: int64
```

Reviews vazias após a limpeza:

0

Distribuição das classes:

```
label
1    25000
0    25000
Name: count, dtype: int64
```

Podemos observar 50.000 amostras perfeitamente balanceadas nas classes 0 e 1. Não existem valores vazios ou nulos no dataset.

Vamos observar então os tokens/palavras mais comuns no texto para analisar se existe necessidade de adicionar alguma funcionalidade na função de pré-processamento. Analisando os dados decidimos adicionar a eliminação de tags HTML na função de pré-processamento. Abaixo mostramos apenas o resultado final.

```
[ ]: tokens = " ".join(df_clean["reviews"]).split()
     token_counts = Counter(tokens)

     token_counts.most_common(200)
```

```
[ ]: [('the', 663910),
      ('and', 320683),
      ('a', 320571),
      ('of', 288481),
      ('to', 266928),
      ('is', 210499),
      ('in', 185063),
      ('it', 154901),
      ('i', 152108),
      ('this', 149898),
      ('that', 136535),
      ('was', 95403),
      ('as', 91242),
      ('with', 87003),
      ('for', 86834),
      ('movie', 85220),
      ('but', 81777),
      ('film', 76026),
      ('on', 66999),
      ('not', 60051),
      ('you', 59816),
      ('are', 58258),
      ('his', 57305),
      ('have', 55111),
      ('be', 52917),
      ('he', 51962),
      ('one', 51475),
      ('its', 49118),
      ('at', 46541),
      ('all', 45705),
      ('by', 44174),
      ('an', 42830),
      ('they', 41548),
      ('from', 40373),
      ('who', 40188),
      ('so', 39637),
      ('like', 39061),
      ('or', 34950),
      ('just', 34872),
      ('her', 34489),
      ('about', 34055),
      ('if', 33590),
      ('out', 33050),
      ('has', 32927),
      ('some', 31004),
      ('there', 30839),
      ('what', 30317),
```

('good', 28912),  
('when', 27892),  
('more', 27869),  
('very', 27598),  
('my', 24762),  
('even', 24581),  
('no', 24459),  
('up', 24457),  
('would', 24036),  
('time', 23958),  
('she', 23894),  
('only', 23097),  
('which', 23046),  
('really', 23010),  
('their', 22699),  
('see', 22639),  
('story', 22510),  
('were', 22233),  
('had', 22029),  
('can', 21894),  
('me', 21219),  
('well', 19218),  
('than', 19177),  
('we', 19119),  
('much', 19107),  
('been', 18301),  
('get', 18243),  
('other', 18049),  
('will', 18028),  
('bad', 17988),  
('do', 17964),  
('great', 17934),  
('also', 17857),  
('into', 17817),  
('people', 17709),  
('because', 17603),  
('how', 17465),  
('him', 17390),  
('most', 17279),  
('first', 17207),  
('dont', 16971),  
('them', 15888),  
('then', 15756),  
('movies', 15716),  
('made', 15558),  
('films', 15536),  
('way', 15364),

('make', 15328),  
('could', 15173),  
('too', 15083),  
('any', 15018),  
('characters', 14900),  
('after', 14782),  
('think', 14272),  
('watch', 13767),  
('many', 13390),  
('seen', 13263),  
('two', 13081),  
('character', 13080),  
('being', 13065),  
('never', 12881),  
('love', 12685),  
('acting', 12595),  
('plot', 12550),  
('did', 12510),  
('where', 12435),  
('best', 12429),  
('know', 12380),  
('little', 12351),  
('show', 12236),  
('life', 12000),  
('ever', 11746),  
('does', 11565),  
('your', 11387),  
('better', 11296),  
('off', 11025),  
('end', 10895),  
('still', 10753),  
('over', 10745),  
('scene', 10725),  
('say', 10688),  
('these', 10687),  
('here', 10426),  
('man', 10423),  
('scenes', 10360),  
('why', 10345),  
('while', 10278),  
('such', 9942),  
('something', 9910),  
('go', 9696),  
('through', 9601),  
('should', 9589),  
('im', 9477),  
('those', 9360),

('back', 9334),  
('watching', 9079),  
('real', 9072),  
('thing', 9042),  
('now', 9018),  
('doesnt', 8902),  
('actors', 8869),  
('didnt', 8848),  
('years', 8812),  
('though', 8563),  
('funny', 8543),  
('another', 8517),  
('actually', 8432),  
('before', 8418),  
('makes', 8293),  
('nothing', 8263),  
('work', 8216),  
('find', 8195),  
('look', 8156),  
('going', 8060),  
('same', 8053),  
('lot', 8036),  
('new', 7985),  
('every', 7934),  
('few', 7933),  
('old', 7791),  
('us', 7758),  
('part', 7714),  
('again', 7702),  
('director', 7540),  
('cant', 7518),  
('thats', 7353),  
('things', 7307),  
('want', 7277),  
('cast', 7253),  
('quite', 7240),  
('pretty', 7197),  
('seems', 7064),  
('around', 7062),  
('down', 7033),  
('world', 6996),  
('young', 6946),  
('got', 6933),  
('take', 6902),  
('however', 6853),  
('fact', 6844),  
('enough', 6841),



```
( 'both', 6756),
( 'horror', 6717),
( 'big', 6695),
( 'thought', 6692),
( 'give', 6660),
( 'ive', 6656),
( 'own', 6555),
( 'may', 6552),
( 'between', 6531),
( 'long', 6396),
( 'without', 6388),
( 'right', 6367)]
```

### 1.1.5 Definindo MAX\_LENGTH

Agora que chegamos numa limpeza aparentemente razoável, vamos ver o tamanho das sentenças. Primeiro definimos uma coluna `length` no dataset `df_clean` para analisar sua distribuição numérica.

```
[10]: df_clean["length"] = df_clean["reviews"].str.split().str.len()
```

Criamos funções auxiliares para descrever a variável numérica.

```
[11]: def br_number(value, decimals=0):
    """
    Formata um número para o formato brasileiro com a quantidade de casas
    ↪decimais desejada.
    """
    formatted = f"{value:,.{decimals}f}"
    return formatted.replace(",", "X").replace(".", ",").replace("X", ".")
```

```
[12]: def descrever_variavel_numerica(y, nome="variável", decimals=0):
    """
    Descreve uma variável numérica com percentis e assimetria.
    """
    percentis = [0, 0.01, 0.05, 0.10, 0.25, 0.50, 0.75, 0.90, 0.95, 0.99, 1.00]

    print(f"=== Percentis de {nome} ===")

    for p in percentis:
        valor = y.quantile(p)
        qtd_acumulada = (y <= valor).sum()

        print(
            f"P {int(p * 100):>3}%:"
            f" {br_number(valor, decimals):>15}"
            f" | qtd acumulada: {qtd_acumulada:>5}"
        )
```

```

print()

print(f"=== Assimetria de {nome} ===")
print(f"{'Média':<10}: {br_number(y.mean(), decimals):>15}")
print(f"{'Mediana':<10}: {br_number(y.median(), decimals):>15}")
print(f"{'Mínimo':<10}: {br_number(y.min(), decimals):>15}")
print(f"{'Máximo':<10}: {br_number(y.max(), decimals):>15}")
print(f"{'Skewness':<10}: {br_number(y.skew(), 2):>15}")
print()

```

E utilizamos as funções definidas para descrever o tamanho das sentenças:

```

[13]: descrever_variavel_numerica(
        df_clean["length"],
        nome="tamanho das reviews",
        decimals=0
    )

```

```

=== Percentis de tamanho das reviews ===
P 0%:          4 | qtd acumulada:    1
P 1%:         41 | qtd acumulada:   537
P 5%:         62 | qtd acumulada:  2528
P 10%:        88 | qtd acumulada:  5033
P 25%:       125 | qtd acumulada: 12746
P 50%:       171 | qtd acumulada: 25117
P 75%:       276 | qtd acumulada: 37507
P 90%:       445 | qtd acumulada: 45019
P 95%:       581 | qtd acumulada: 47503
P 99%:       892 | qtd acumulada: 49500
P 100%:      2.459 | qtd acumulada: 50000

```

```

=== Assimetria de tamanho das reviews ===
Média      :          228
Mediana    :          171
Mínimo     :           4
Máximo     :         2.459
Skewness   :         2,17

```

A distribuição do tamanho das reviews apresenta assimetria à direita. A mediana é de 171 palavras, enquanto a média é de 228 palavras, diferença pode ser explicada pela presença de reviews mais longas na distribuição. O coeficiente de assimetria (skewness) de 2,17 confirma esse comportamento. Em relação aos percentis, 75% das reviews possuem até 276 palavras, 90% possuem até 445 palavras e 95% possuem até 581 palavras. Embora a maior review contenha 2.459 palavras, os valores extremos representam uma parcela pequena do conjunto: apenas 1% das reviews ultrapassa 892 palavras. Esses resultados indicam que a definição do tamanho máximo da entrada deve priorizar os percentis intermediários, evitando dimensionar o modelo com base nos casos extremos e reduzindo a quantidade desnecessária de preenchimento (padding) nas sequências menores.

Para confirmar isso vamos produzir um gráfico de distribuição da quantidade de palavras no dataset. Primeiro definimos funções auxiliares para criar o gráfico.

```
[14]: def formatar_milhoes_br(x, pos):  
    '''  
    Formata um número para o formato brasileiro com a quantidade de casas_  
    ↳decimais desejada.  
    '''  
    return f"{x / 1_000_000:.1f}".replace(".", ",")  
  
def plotar_histograma_com_percentis(  
    y,  
    nome="variável",  
    limite_percentil=0.99,  
    percentis_marcados=[0.25, 0.50, 0.75, 0.90, 0.95, 0.99],  
    bins=50,  
    usar_milhoes=True  
):  
    '''  
    Plota um histograma dos dados, juntamente com os percentis que desejamos_  
    ↳marcar no grafico.  
    '''  
    limite = y.quantile(limite_percentil)  
    y_filtrado = y[y <= limite]  
  
    plt.figure(figsize=(12, 6))  
    plt.hist(y_filtrado, bins=bins, edgecolor="black", alpha=0.7)  
  
    if usar_milhoes:  
        plt.gca().xaxis.set_major_formatter(FuncFormatter(formatar_milhoes_br))  
        xlabel = f"{nome} em milhões"  
    else:  
        xlabel = nome  
  
    altura_max = plt.ylim()[1]  
  
    for p in percentis_marcados:  
        valor = y.quantile(p)  
  
        if valor <= limite:  
            plt.axvline(valor, linestyle="--", linewidth=1.5, alpha=0.8)  
            plt.text(  
                valor,  
                altura_max * 0.95,  
                f"P{int(p * 100)}",  
                rotation=90,
```

```

        ha="center",
        va="top"
    )

    plt.title(f"Distribuição de {nome} até o percentil {int(limite_percentil * 100)}")
    plt.xlabel(xlabel)
    plt.ylabel("Frequência")
    plt.grid(True, axis="y", alpha=0.3)
    plt.show()

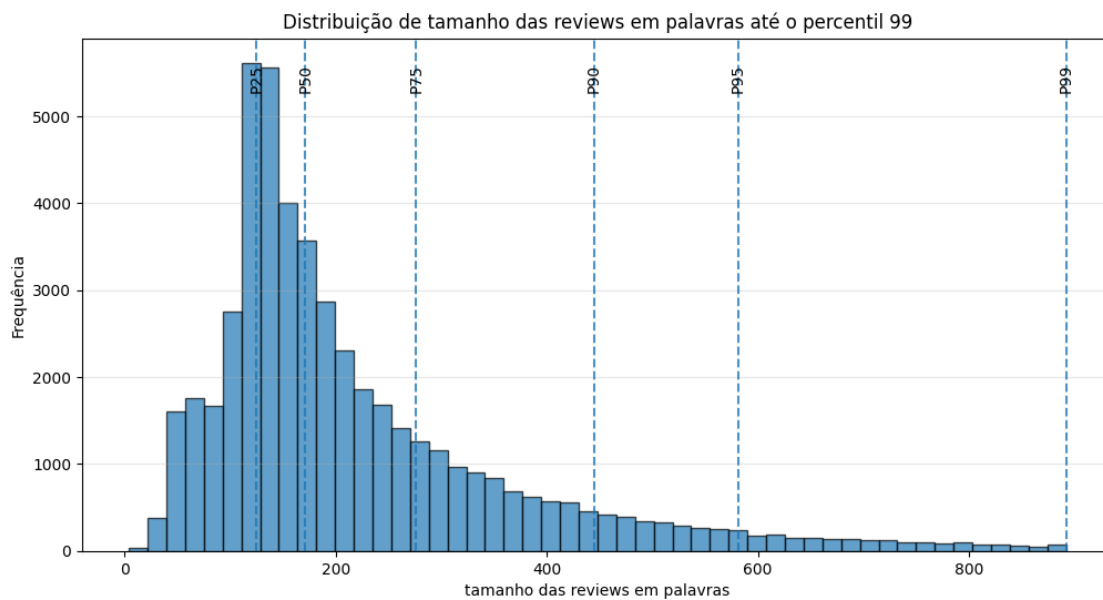
```

Depois utilizamos a função para gerar o gráfico com a distribuição da quantidade de palavras por review do dataset.

```

[15]: plotar_histograma_com_percentis(
    df_clean["length"],
    nome="tamanho das reviews em palavras",
    limite_percentil=0.99,
    percentis_marcados=[0.25, 0.50, 0.75, 0.90, 0.95, 0.99],
    bins=50,
    usar_milhoes=False
)

```



O histograma evidencia uma distribuição assimétrica à direita, com maior concentração de reviews entre aproximadamente 100 e 300 palavras e uma redução progressiva da frequência à medida que o tamanho aumenta. A presença de uma cauda longa indica que existem reviews significativamente maiores que a maioria, embora representem uma parcela reduzida do conjunto de dados.

Os percentis destacados no gráfico permitem observar esse comportamento de forma mais precisa. Enquanto 50% das reviews possuem até 171 palavras e 75% possuem até 276 palavras, o percentil 95 corresponde a 581 palavras. Isso significa que 95% das reviews possuem tamanho igual ou inferior a esse valor.

Dessa forma, foi escolhido o percentil 95 como referência para definir o tamanho máximo das entradas do modelo. Essa escolha permite preservar a maior parte do conteúdo textual disponível, ao mesmo tempo em que evita dimensionar todas as entradas com base em casos extremos. Reviews maiores que esse limite serão truncadas, enquanto reviews menores receberão preenchimento até o tamanho definido. Esse critério representa um equilíbrio entre retenção de informação e custo computacional.

```
[16]: MAX_LENGTH = int(df_clean["length"].quantile(0.95))

print(MAX_LENGTH)
```

581

### 1.1.6 Definindo VOCAB\_SIZE

Agora vamos avaliar a cobertura dos tokens em função do tamanho do vocabulário. O que queremos responder/definir é o seguinte: qual o tamanho do vocabulário nos permite extrair o máximo desempenho do dataset sem comprometer demais os recursos computacionais necessários para processamento?

Para isso vamos tentar mostrar, em termos de cobertura de vocabulário, qual o ganho marginal se obtém ao se aumentar o tamanho do vocabulário. Buscaremos identificar um ponto onde aumentar o tamanho do vocabulário gera um incremento marginal na cobertura.

Para isso, primeiro vamos contar a quantidade de tokens total e quantidade de tokens únicos do dataset.

```
[17]: # Conta todas as ocorrências de palavras no dataset pré-processado
token_counts = Counter(
    token
    for review in df_clean["reviews"]
    for token in review.split()
)

# Ordena os tokens do mais frequente para o menos frequente
frequencias_ordenadas = sorted(token_counts.values(), reverse=True)

total_ocorrencias = sum(frequencias_ordenadas)
total_tokens_unicos = len(frequencias_ordenadas)

print(f"Total de ocorrências de palavras: {br_number(total_ocorrencias)}")
print(f"Quantidade de palavras distintas: {br_number(total_tokens_unicos)}")
```

Total de ocorrências de palavras: 11.395.898

Quantidade de palavras distintas: 167.168

E definir alguns valores-alvo de vocab\_size para averiguar a cobertura.

```
[18]: vocab_sizes = [  
    1000,  
    2000,  
    5000,  
    10000,  
    15000,  
    20000,  
    30000,  
    50000  
]
```

Agora, para verificar a cobertura, vamos criar um dataset contendo, para cada um dos vocab\_size propostos, a quantidade e percentual de ocorrências cobertas, e a quantidade e percentual de ocorrências fora do vocabulário (OOV: Out Of Vocabulary).

```
[19]: resultados_vocabulario = []  
  
for vocab_size in vocab_sizes:  
    tamanho_efetivo = min(vocab_size, total_tokens_unicos)  
  
    ocorrencias_cobertas = sum(frequencias_ordenadas[:tamanho_efetivo])  
    ocorrencias_oov = total_ocorrencias - ocorrencias_cobertas  
  
    resultados_vocabulario.append({  
        "VOCAB_SIZE": vocab_size,  
        "palavras incluídas": tamanho_efetivo,  
        "ocorrências cobertas": ocorrencias_cobertas,  
        "cobertura (%)": ocorrencias_cobertas / total_ocorrencias * 100,  
        "ocorrências OOV": ocorrencias_oov,  
        "OOV (%)": ocorrencias_oov / total_ocorrencias * 100  
    })  
  
df_vocabulario = pd.DataFrame(resultados_vocabulario)  
  
df_vocabulario.style.format({  
    "VOCAB_SIZE": "{:,.0f}",  
    "palavras incluídas": "{:,.0f}",  
    "ocorrências cobertas": "{:,.0f}",  
    "cobertura (%)": "{:.2f}",  
    "ocorrências OOV": "{:,.0f}",  
    "OOV (%)": "{:.2f}"  
})
```

```
[19]: <pandas.io.formats.style.Styler at 0x14e8d8170>
```

Os resultados mostram que a cobertura cresce rapidamente até aproximadamente 10.000 palavras e passa a apresentar ganhos marginais progressivamente menores a partir desse ponto. Com 10.000

palavras, o vocabulário cobre 93,55% das ocorrências do corpus, deixando 6,45% como tokens fora do vocabulário. Ao aumentar para 15.000, a cobertura sobe para 95,44%, um ganho de 1,89 ponto percentual. Com 20.000, a cobertura chega a 96,53%, representando ganho adicional de apenas 1,09 ponto percentual. A partir daí, o crescimento se torna ainda menos expressivo: usar 50.000 palavras eleva a cobertura para 98,68%, mas exige uma matriz de embeddings cinco vezes maior que a configuração com 10.000.

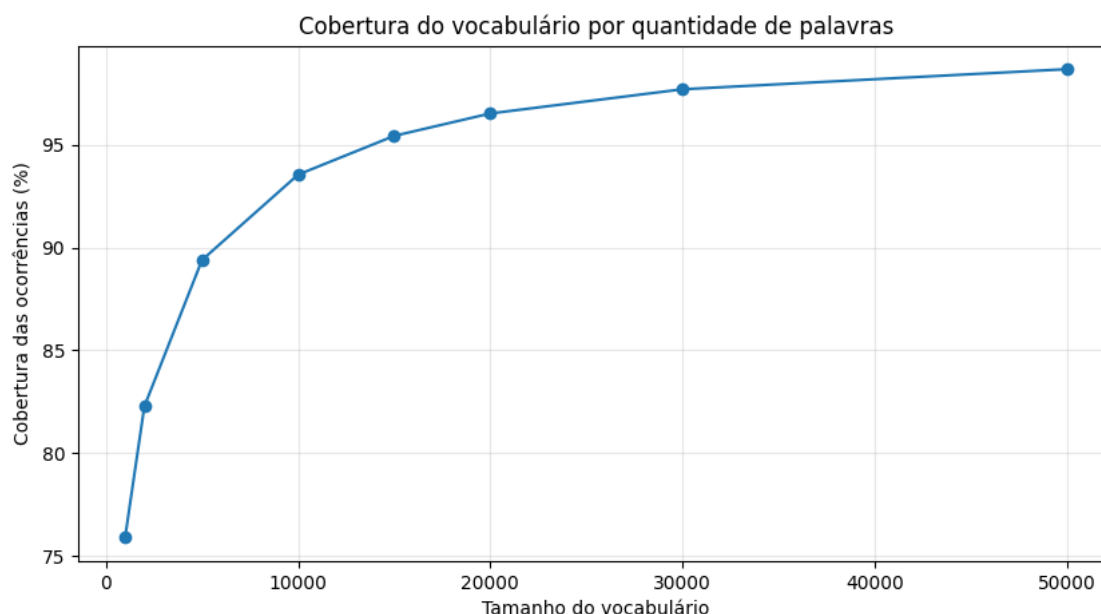
Vamos visualizar essa curva num gráfico para definir o tamanho do vocabulario que será utilizado.

```
[20]: plt.figure(figsize=(10, 5))

plt.plot(
    df_vocabulario["VOCAB_SIZE"],
    df_vocabulario["cobertura (%)"],
    marker="o"
)

plt.title("Cobertura do vocabulário por quantidade de palavras")
plt.xlabel("Tamanho do vocabulário")
plt.ylabel("Cobertura das ocorrências (%)")
plt.grid(True, alpha=0.3)

plt.show()
```



O gráfico indica um ponto de inflexão entre 10.000 e 15.000 palavras. Considerando o equilíbrio entre cobertura do vocabulário, custo computacional e quantidade de parâmetros, 15.000 nos parece uma escolha mais adequada para nosso caso. Esse valor cobre pouco mais de 95% das ocorrências e reduz a proporção de tokens desconhecidos para 4,56%, sem aumentar excessivamente a complexidade do

modelo.

```
[21]: VOCAB_SIZE = 15000
```

### 1.1.7 2º Passo:

- Realize a transformação da variável alvo;
- Defina o objeto da vetorização dos dados textuais com a função `TextVectorization`:
  - Faça operações *regex* customizadas, caso seja necessário.
  - Defina o tamanho do vocabulário (`vocab_size`), o tamanho da entrada/janela do modelo (`max_len`), e a dimensão do vetor *embedding* (`embed_dim`);
  - Iremos realizar a vetorização (indexação) dentro da rede neural, ou seja, o objeto da vetorização irá processar os dados após a camada `layers.Input()`.
- Divida os dados em treinamento (90%) e teste (10%). Os dados de validação serão divididos na função `model.fit()`.

A definição das operações de limpeza dos dados (`Preprocessor`), do tamanho de entrada do modelo (`MAX_LENGTH=581`) e do tamanho do vocabulário (`VOCAB_SIZE=15000`) foram realizadas e justificadas no passo anterior.

Ja o número de dimensões do embedding não pode ser definido exclusivamente em função do vocabulário. Ele deve ser tratado como um hiperparâmetro do modelo e deve ser definido em função do desempenho do modelo. Poucas dimensões podem não ser suficientes para representar adequadamente a relação entre as palavras; muitas dimensões podem gerar overfitting; portanto, vamos iniciar com um valor mais ou menos arbitrário, moderado, que nos permita testar o modelo: `EMBED_DIM = 100`. Inclusive, por que esse valor inicial nos permitirá utilizar os embeddings pre-treinados do GloVe com mais facilidade, caso seja desejado.

```
[22]: #EMBED_DIM = 100

vectorizer = layers.TextVectorization(
    name="TextVectorizer",
    max_tokens=VOCAB_SIZE,
    output_sequence_length=MAX_LENGTH,
    standardize=Preprocessor,
    output_mode="int",
)
```

Agora definimos o conjunto de treino e teste do nosso dataset, tomando o cuidado de manter a estratificação das classes.

```
[23]: TEST_SIZE = 0.10

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=TEST_SIZE,
    random_state=RANDOM_STATE,
    stratify=y)
```



```
)
```

E confirmamos que a estratificação foi corretamente mantida.

```
[24]: def comparar_distribuicao_classes(
    y_total,
    y_treino,
    y_avaliacao,
    nome_treino="Treino",
    nome_avaliacao="Teste"
):
    distribuicao_classes = pd.DataFrame({
        "Total - qtd": y_total.value_counts().sort_index(),
        "Total - %": y_total.value_counts(normalize=True).sort_index() * 100,
        f"{nome_treino} - qtd": y_treino.value_counts().sort_index(),
        f"{nome_treino} - %": y_treino.value_counts(normalize=True).
↪sort_index() * 100,
        f"{nome_avaliacao} - qtd": y_avaliacao.value_counts().sort_index(),
        f"{nome_avaliacao} - %": y_avaliacao.value_counts(normalize=True).
↪sort_index() * 100
    })

    distribuicao_classes.index.name = "Classe"

    return distribuicao_classes.style.format({
        "Total - qtd": "{:,.0f}",
        "Total - %": "{:.2f}%",
        f"{nome_treino} - qtd": "{:,.0f}",
        f"{nome_treino} - %": "{:.2f}%",
        f"{nome_avaliacao} - qtd": "{:,.0f}",
        f"{nome_avaliacao} - %": "{:.2f}%"
    })
```

```
[25]: comparar_distribuicao_classes(
    y_total=y,
    y_treino=y_train,
    y_avaliacao=y_test
)
```

```
[25]: <pandas.io.formats.style.Styler at 0x14f2d9be0>
```

Dessa forma, temos os dados definidos em `X_train`, `X_test`, `y_train` e `y_test` mantendo a estratificação das classes e com o percentual de 10% para teste conforme solicitado.

### 1.1.8 3º Passo:

- Desenvolva a primeira versão da **arquitetura neural**:
  - Já sabemos o tamanho da entrada, o tamanho da saída e sua função de ativação:  
`x_in = layers.Input(shape=(1, ), dtype=tf.string)`

- A camada `Embedding` irá retornar um *array* de formato (`max_len`, `embed_dim`), assim, precisamos torná-lo unidimensional para poder ser acoplado à próxima camada. Para isso, utilize:
  - \* Uma camada de `Flatten`;
  - \* Ou uma camada de `GlobalAveragePooling1D`.
- Compile o modelo com um otimizador (`SGR`, `RMSprop`, `Adam`);
- Defina a função de perda e métrica de acordo com a natureza do problema.

Primeiro vamos dividir os dados de treinamento em treino (tune) e validação, novamente com percentual de 10%.

```
[26]: VALIDATION_SIZE = 0.10

X_tune_train, X_val, y_tune_train, y_val = train_test_split(
    X_train,
    y_train,
    test_size=VALIDATION_SIZE,
    random_state=RANDOM_STATE,
    stratify=y_train
)

comparar_distribuicao_classes(
    y_total=y_train,
    y_treino=y_tune_train,
    y_avaliacao=y_val,
    nome_treino="Treino para busca",
    nome_avaliacao="Validação"
)
```

```
[26]: <pandas.io.formats.style.Styler at 0x12b2919a0>
```

Preparamos os dados para executar o treinamento.

```
[27]: X_tune_train_array = tf.convert_to_tensor(
    X_tune_train.astype(str).to_numpy().reshape(-1, 1),
    dtype=tf.string
)

X_val_array = tf.convert_to_tensor(
    X_val.astype(str).to_numpy().reshape(-1, 1),
    dtype=tf.string
)

y_tune_train_array = tf.convert_to_tensor(
    y_tune_train.to_numpy(),
    dtype=tf.float32
)

y_val_array = tf.convert_to_tensor(
```

```
y_val.to_numpy(),
dtype=tf.float32
)
```

```
[28]: print(f"Treino para busca : {X_tune_train_array.shape}")
      print(f"Validação       : {X_val_array.shape}")
      print(f"Teste reservado  : {len(X_test)}")
```

```
Treino para busca : (40500, 1)
Validação         : (4500, 1)
Teste reservado   : 5000
```

E criamos o vetorizador utilizando os parametros que definimos anteriormente (VOCAB\_SIZE=15.000 e MAX\_LENGTH=581).

```
[29]: vectorizer = layers.TextVectorization(
      standardize=Preprocessor,
      max_tokens=VOCAB_SIZE,
      output_mode="int",
      output_sequence_length=MAX_LENGTH
    )

    vectorizer.adapt(X_tune_train_array)

    print(f"Tamanho efetivo do vocabulário: {vectorizer.vocabulary_size()}")
```

```
Tamanho efetivo do vocabulário: 15000
```

Ao invés de executar e treinar alguns poucos modelos, vamos adotar uma abordagem mais sistematizada para definição do modelo. Trabalharemos com RandomSearch e GridSearch e trataremos alguns hiperparâmetros do modelo com essas buscas. Para isso, vamos utilizar Keras Tuner com uma busca randômica preliminar e os seguintes hiperparâmetros:

- embed\_dim = [50, 100, 200]
- dense\_units = [16, 32, 64]
- dropout\_rate = [0.0, 0.2, 0.4]
- optimizer = ["adam", "rmsprop", "sgd"]
- learning\_rate = [1e-2, 1e-3, 1e-4]

Isso nos dá um espaço de busca com 243 combinações possíveis. Vamos utilizar early stopping com patience = 3 e apenas 12 execuções. Isso nos dará uma cobertura de aproximadamente 5% do espaço, o que é baixo. Mas como é uma busca preliminar, vamos analisar os resultados obtidos para definir uma segunda busca com as configurações mais promissoras posteriormente.

```
[30]: def build_model(hp):

      embed_dim = hp.Choice(
          "embed_dim",
          values=[50, 100, 200]
      )
```

```

dense_units = hp.Choice(
    "dense_units",
    values=[16, 32, 64]
)

dropout_rate = hp.Choice(
    "dropout_rate",
    values=[0.0, 0.2, 0.4]
)

optimizer_name = hp.Choice(
    "optimizer",
    values=["adam", "rmsprop", "sgd"]
)

learning_rate = hp.Choice(
    "learning_rate",
    values=[1e-2, 1e-3, 1e-4]
)

x_in = layers.Input(
    shape=(1,),
    dtype=tf.string,
    name="review"
)

x = vectorizer(x_in)

x = layers.Embedding(
    input_dim=vectorizer.vocabulary_size(),
    output_dim=embed_dim,
    mask_zero=True,
    name="embedding"
)(x)

x = layers.GlobalAveragePooling1D(
    name="global_average_pooling"
)(x)

x = layers.Dense(
    units=dense_units,
    activation="relu",
    name="hidden"
)(x)

x = layers.Dropout(

```

```

        rate=dropout_rate,
        name="dropout"
    )(x)

    y_out = layers.Dense(
        units=1,
        activation="sigmoid",
        name="output"
    )(x)

    model = tf.keras.Model(
        inputs=x_in,
        outputs=y_out
    )

    if optimizer_name == "adam":
        optimizer = tf.keras.optimizers.Adam(
            learning_rate=learning_rate
        )
    elif optimizer_name == "rmsprop":
        optimizer = tf.keras.optimizers.RMSprop(
            learning_rate=learning_rate
        )
    else:
        optimizer = tf.keras.optimizers.SGD(
            learning_rate=learning_rate
        )

    model.compile(
        optimizer=optimizer,
        loss="binary_crossentropy",
        metrics=[
            tf.keras.metrics.BinaryAccuracy(name="accuracy")
        ]
    )

    return model

```

Definimos nosso random search e exibimos um sumário do espaço de busca.

```

[31]: tf.keras.utils.set_random_seed(RANDOM_STATE)

tuner = kt.RandomSearch(
    hypermodel=build_model,
    objective="val_accuracy",
    max_trials=12,
    executions_per_trial=1,

```

```

        overwrite=True,
        directory="keras_tuner_results",
        project_name="imdb_sentiment_initial",
        seed=RANDOM_STATE
    )

    tuner.search_space_summary()

```

```

Search space summary
Default search space size: 5
embed_dim (Choice)
{'default': 50, 'conditions': [], 'values': [50, 100, 200], 'ordered': True}
dense_units (Choice)
{'default': 16, 'conditions': [], 'values': [16, 32, 64], 'ordered': True}
dropout_rate (Choice)
{'default': 0.0, 'conditions': [], 'values': [0.0, 0.2, 0.4], 'ordered': True}
optimizer (Choice)
{'default': 'adam', 'conditions': [], 'values': ['adam', 'rmsprop', 'sgd'],
'ordered': False}
learning_rate (Choice)
{'default': 0.01, 'conditions': [], 'values': [0.01, 0.001, 0.0001], 'ordered':
True}

```

Definimos o early stopping e executamos a busca.

```

[32]: early_stopping = tf.keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=3,
        restore_best_weights=True
    )

    tensorboard_callback = tf.keras.callbacks.TensorBoard(
        log_dir="keras_tuner_logs"
    )

    tuner.search(
        X_tune_train_array,
        y_tune_train_array,
        validation_data=(X_val_array, y_val_array),
        epochs=50,
        batch_size=64,
        callbacks=[
            early_stopping,
            tensorboard_callback
        ],
        shuffle=True,
        verbose=1
    )

```

Trial 12 Complete [00h 00m 55s]  
val\_accuracy: 0.895555558204651

Best val\_accuracy So Far: 0.9017778038978577  
Total elapsed time: 00h 29m 01s

Por fim, montamos um dataset com os resultados de cada uma das 12 execuções.

```
[35]: resultados_trials = []

for trial in tuner.oracle.trials.values():
    resultados_trials.append({
        "trial_id": trial.trial_id,
        "status": trial.status,
        "val_accuracy": trial.score,
        "melhor_epoca": trial.best_step + 1 if trial.best_step is not None else
        None,
        **trial.hyperparameters.values
    })

df_trials = (
    pd.DataFrame(resultados_trials)
    .sort_values("val_accuracy", ascending=False)
    .reset_index(drop=True)
)

df_trials
```

```
[35]:
```

|    | trial_id | status    | val_accuracy | melhor_epoca | embed_dim | dense_units | \ |
|----|----------|-----------|--------------|--------------|-----------|-------------|---|
| 0  | 08       | COMPLETED | 0.901778     | 16           | 50        | 32          |   |
| 1  | 00       | COMPLETED | 0.901111     | 15           | 100       | 16          |   |
| 2  | 02       | COMPLETED | 0.900222     | 10           | 200       | 32          |   |
| 3  | 07       | COMPLETED | 0.895778     | 1            | 200       | 64          |   |
| 4  | 11       | COMPLETED | 0.895556     | 2            | 50        | 32          |   |
| 5  | 03       | COMPLETED | 0.893333     | 2            | 50        | 32          |   |
| 6  | 06       | COMPLETED | 0.886444     | 4            | 50        | 16          |   |
| 7  | 04       | COMPLETED | 0.885333     | 2            | 200       | 32          |   |
| 8  | 01       | COMPLETED | 0.884889     | 3            | 100       | 32          |   |
| 9  | 09       | COMPLETED | 0.878222     | 3            | 50        | 32          |   |
| 10 | 05       | COMPLETED | 0.636000     | 50           | 50        | 64          |   |
| 11 | 10       | COMPLETED | 0.565556     | 50           | 100       | 64          |   |

|   | dropout_rate | optimizer | learning_rate |
|---|--------------|-----------|---------------|
| 0 | 0.4          | adam      | 0.0001        |
| 1 | 0.2          | adam      | 0.0001        |
| 2 | 0.4          | adam      | 0.0001        |
| 3 | 0.0          | adam      | 0.0010        |
| 4 | 0.2          | adam      | 0.0010        |

|    |     |         |        |
|----|-----|---------|--------|
| 5  | 0.0 | adam    | 0.0010 |
| 6  | 0.4 | rmsprop | 0.0010 |
| 7  | 0.2 | rmsprop | 0.0100 |
| 8  | 0.0 | rmsprop | 0.0100 |
| 9  | 0.0 | adam    | 0.0100 |
| 10 | 0.4 | sgd     | 0.0100 |
| 11 | 0.2 | sgd     | 0.0010 |

Nosso random search preliminar nos permite tirar algumas conclusões e direcionar uma segunda busca:

1. **Otimizador:** Adam foi claramente superior: para a próxima rodada podemos descartar sgd e rmsprop.
2. **Learning Rate:**  $1e-4$  apresentou melhores resultados, mas acreditamos que podemos explorar alguns valores próximos.
3. **Dimensão de Embeddings:** `embed_dim` maiores não geraram melhores resultados. Na verdade a diferença entre eles foi praticamente irrelevante. Vamos trabalhar apenas `embed_dim = 50` na próxima etapa.
4. **Tamanho da Camada Densa:** embora `dense_units = 32` tenha gerado a melhor execução, não houve uma clara dominância dessa configuração sobre as demais. Vamos seguir explorando esse parâmetro na seguinte rodada.
5. **Dropout:** também não houve clara dominância de alguma configuração nesse parâmetro. Entretanto, podemos observar que o valor de `dropout = 0.0` não parece muito promissor com o otimizador Adam. Portanto, vamos descartar essa opção na próxima rodada.

#### 1.1.9 4º Passo:

- Treine o modelo com o método `.fit()`, definindo as variáveis:
  - `validation_split`: porcentagem de dados para validação;
  - `epochs`: número de épocas para o treinamento;
  - `batch_size`: tamanho do lote da amostragem dos dados de entrada.

Considerando as observações anteriores, vamos definir uma segunda rodada com as seguintes configurações:

- `embed_dim = [50]`
- `dense_units = [16, 32, 64]`
- `dropout_rate = [0.2, 0.3, 0.4, 0.5]`
- `optimizer = ["adam"]`
- `learning_rate = [5e-5, 1e-4, 2e-4]`

Como agora temos um espaço mais reduzido, 36 opções apenas, vamos modificar a estratégia e executar um GridSearch para explorar todo o espaço de busca. Vamos manter early stopping, mas ajustando patience para 4, já que estamos testando learning rates menores, o que pode levar os modelos a aprenderem mais devagar. Redefinimos a função `build_model` para operar com o novo espaço de busca.

```
[36]: def build_model(hp):
      embed_dim = 50
```



```

dense_units = hp.Choice(
    "dense_units",
    values=[16, 32, 64]
)

dropout_rate = hp.Choice(
    "dropout_rate",
    values=[0.2, 0.3, 0.4, 0.5]
)

learning_rate = hp.Choice(
    "learning_rate",
    values=[5e-5, 1e-4, 2e-4]
)

x_in = layers.Input(
    shape=(1,),
    dtype=tf.string,
    name="review"
)

x = vectorizer(x_in)

x = layers.Embedding(
    input_dim=vectorizer.vocabulary_size(),
    output_dim=embed_dim,
    mask_zero=True,
    name="embedding"
)(x)

x = layers.GlobalAveragePooling1D(
    name="global_average_pooling"
)(x)

x = layers.Dense(
    units=dense_units,
    activation="relu",
    name="hidden"
)(x)

x = layers.Dropout(
    rate=dropout_rate,
    name="dropout"
)(x)

y_out = layers.Dense(

```

```

        units=1,
        activation="sigmoid",
        name="output"
    )(x)

    model = tf.keras.Model(
        inputs=x_in,
        outputs=y_out
    )

    optimizer = tf.keras.optimizers.Adam(
        learning_rate=learning_rate
    )

    model.compile(
        optimizer=optimizer,
        loss="binary_crossentropy",
        metrics=[
            tf.keras.metrics.BinaryAccuracy(name="accuracy")
        ]
    )

    return model

```

Agora definimos o tuner (GridSearch) e early stopping com patience = 4 pois agora estamos trabalhando com valores de learning rate menores. Por fim executamos a busca.

```

[37]: tuner = kt.GridSearch(
    hypermodel=build_model,
    objective="val_accuracy",
    max_trials=36,
    executions_per_trial=1,
    overwrite=True,
    directory="keras_tuner_results",
    project_name="imdb_sentiment_refined"
)

early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor="val_accuracy",
    mode="max",
    patience=4,
    restore_best_weights=True
)

tensorboard_callback = tf.keras.callbacks.TensorBoard(
    log_dir="keras_tuner_logs/imdb_sentiment_refined"
)

```

```
tuner.search(
    X_tune_train_array,
    y_tune_train_array,
    validation_data=(X_val_array, y_val_array),
    epochs=50,
    batch_size=64,
    callbacks=[
        early_stopping,
        tensorboard_callback
    ],
    shuffle=True,
    verbose=1
)
```

Trial 36 Complete [00h 02m 13s]  
val\_accuracy: 0.9006666541099548

Best val\_accuracy So Far: 0.9026666879653931  
Total elapsed time: 02h 27m 58s

Agora geramos uma tabela com todos os resultados resumidos.

```
[38]: resultados_grid = []

for trial in tuner.oracle.trials.values():
    resultados_grid.append({
        "trial_id": trial.trial_id,
        "status": trial.status,
        "val_accuracy": trial.score,
        "melhor_epoca": trial.best_step + 1 if trial.best_step is not None else_
        ↪None,
        "dense_units": trial.hyperparameters.get("dense_units"),
        "dropout_rate": trial.hyperparameters.get("dropout_rate"),
        "learning_rate": trial.hyperparameters.get("learning_rate")
    })

df_grid = (
    pd.DataFrame(resultados_grid)
    .sort_values("val_accuracy", ascending=False)
    .reset_index(drop=True)
)

df_grid.insert(
    0,
    "posição",
    range(1, len(df_grid) + 1)
)
```

```
df_grid
```

```
[38]:
```

|    | posição | trial_id | status    | val_accuracy | melhor_epoca | dense_units | \ |
|----|---------|----------|-----------|--------------|--------------|-------------|---|
| 0  | 1       | 16       | COMPLETED | 0.902667     | 14           | 32          |   |
| 1  | 2       | 09       | COMPLETED | 0.902667     | 41           | 16          |   |
| 2  | 3       | 10       | COMPLETED | 0.902444     | 19           | 16          |   |
| 3  | 4       | 34       | COMPLETED | 0.902000     | 14           | 64          |   |
| 4  | 5       | 24       | COMPLETED | 0.902000     | 24           | 64          |   |
| 5  | 6       | 11       | COMPLETED | 0.901778     | 9            | 16          |   |
| 6  | 7       | 19       | COMPLETED | 0.901556     | 17           | 32          |   |
| 7  | 8       | 01       | COMPLETED | 0.901556     | 17           | 16          |   |
| 8  | 9       | 13       | COMPLETED | 0.901556     | 15           | 32          |   |
| 9  | 10      | 27       | COMPLETED | 0.901333     | 30           | 64          |   |
| 10 | 11      | 02       | COMPLETED | 0.901333     | 11           | 16          |   |
| 11 | 12      | 18       | COMPLETED | 0.901111     | 32           | 32          |   |
| 12 | 13      | 22       | COMPLETED | 0.901111     | 17           | 32          |   |
| 13 | 14      | 07       | COMPLETED | 0.901111     | 20           | 16          |   |
| 14 | 15      | 31       | COMPLETED | 0.901111     | 12           | 64          |   |
| 15 | 16      | 21       | COMPLETED | 0.901111     | 29           | 32          |   |
| 16 | 17      | 25       | COMPLETED | 0.900889     | 14           | 64          |   |
| 17 | 18      | 06       | COMPLETED | 0.900889     | 38           | 16          |   |
| 18 | 19      | 17       | COMPLETED | 0.900889     | 9            | 32          |   |
| 19 | 20      | 32       | COMPLETED | 0.900889     | 8            | 64          |   |
| 20 | 21      | 23       | COMPLETED | 0.900889     | 8            | 32          |   |
| 21 | 22      | 28       | COMPLETED | 0.900667     | 16           | 64          |   |
| 22 | 23      | 35       | COMPLETED | 0.900667     | 8            | 64          |   |
| 23 | 24      | 20       | COMPLETED | 0.900667     | 9            | 32          |   |
| 24 | 25      | 15       | COMPLETED | 0.900667     | 27           | 32          |   |
| 25 | 26      | 14       | COMPLETED | 0.900667     | 9            | 32          |   |
| 26 | 27      | 12       | COMPLETED | 0.900667     | 25           | 32          |   |
| 27 | 28      | 04       | COMPLETED | 0.900444     | 18           | 16          |   |
| 28 | 29      | 29       | COMPLETED | 0.900222     | 8            | 64          |   |
| 29 | 30      | 30       | COMPLETED | 0.900222     | 24           | 64          |   |
| 30 | 31      | 33       | COMPLETED | 0.900222     | 26           | 64          |   |
| 31 | 32      | 03       | COMPLETED | 0.900000     | 37           | 16          |   |
| 32 | 33      | 00       | COMPLETED | 0.900000     | 28           | 16          |   |
| 33 | 34      | 26       | COMPLETED | 0.899778     | 7            | 64          |   |
| 34 | 35      | 08       | COMPLETED | 0.899778     | 9            | 16          |   |
| 35 | 36      | 05       | COMPLETED | 0.899111     | 8            | 16          |   |

|   | dropout_rate | learning_rate |
|---|--------------|---------------|
| 0 | 0.3          | 0.00010       |
| 1 | 0.5          | 0.00005       |
| 2 | 0.5          | 0.00010       |
| 3 | 0.5          | 0.00010       |
| 4 | 0.2          | 0.00005       |

|    |     |         |
|----|-----|---------|
| 5  | 0.5 | 0.00020 |
| 6  | 0.4 | 0.00010 |
| 7  | 0.2 | 0.00010 |
| 8  | 0.2 | 0.00010 |
| 9  | 0.3 | 0.00005 |
| 10 | 0.2 | 0.00020 |
| 11 | 0.4 | 0.00005 |
| 12 | 0.5 | 0.00010 |
| 13 | 0.4 | 0.00010 |
| 14 | 0.4 | 0.00010 |
| 15 | 0.5 | 0.00005 |
| 16 | 0.2 | 0.00010 |
| 17 | 0.4 | 0.00005 |
| 18 | 0.3 | 0.00020 |
| 19 | 0.4 | 0.00020 |
| 20 | 0.5 | 0.00020 |
| 21 | 0.3 | 0.00010 |
| 22 | 0.5 | 0.00020 |
| 23 | 0.4 | 0.00020 |
| 24 | 0.3 | 0.00005 |
| 25 | 0.2 | 0.00020 |
| 26 | 0.2 | 0.00005 |
| 27 | 0.3 | 0.00010 |
| 28 | 0.3 | 0.00020 |
| 29 | 0.4 | 0.00005 |
| 30 | 0.5 | 0.00005 |
| 31 | 0.3 | 0.00005 |
| 32 | 0.2 | 0.00005 |
| 33 | 0.2 | 0.00020 |
| 34 | 0.4 | 0.00020 |
| 35 | 0.3 | 0.00020 |

Agora que treinamos os 36 modelos, vamos diminuir patience para 3 e treinar novamente apenas os top 6 modelos em termos de acurácia. Decidimos diminuir patience por que não parece que houve ganho significativo no aumento do tempo de espera para atuação do early stopping.

Em seguida vamos gerar graficos de acurácia e loss ao longo das épocas de treinamento e escolher o modelo que será utilizado nos passos seguintes.

Vale lembrar que os resultados numéricos podem ser ligeiramente distintos por conta da nova inicialização dos pesos do modelo.

```
[49]: def retreinar_e_plotar_melhores_modelos(
        tuner,
        build_model,
        X_train,
        y_train,
        X_val,
```

```

y_val,
top_n=3,
epochs=50,
batch_size=64,
patience=4,
random_state=42
):
    """
    Retreina os melhores modelos encontrados pelo tuner e plota seus históricos.

    Para cada modelo, gera dois gráficos lado a lado:
    - acurácia de treino e validação;
    - loss de treino e validação.

    A época com maior val_accuracy é marcada por uma linha vertical pontilhada.

    Todos os gráficos usam a mesma escala no eixo x, de 1 até `epochs`,
    permitindo comparação visual direta entre os modelos.

    Parameters
    -----
    tuner : keras_tuner.Tuner
        Tuner previamente executado.

    build_model : callable
        Função utilizada para construir o modelo a partir dos hiperparâmetros.

    X_train, y_train
        Dados de treino utilizados durante a busca.

    X_val, y_val
        Dados de validação utilizados durante a busca.

    top_n : int
        Quantidade de melhores modelos que serão retreinados.

    epochs : int
        Quantidade máxima de épocas por treinamento.

    batch_size : int
        Quantidade de amostras em cada batch.

    patience : int
        Quantidade de épocas sem melhora antes da interrupção antecipada.

    random_state : int
        Semente utilizada para aumentar a reprodutibilidade.

```

*Returns*

-----

*dict*

*Modelos, históricos e hiperparâmetros das novas execuções.*

"""

```
melhores_trials = tuner.oracle.get_best_trials(num_trials=top_n)
```

```
resultados_retreino = {}
```

```
fig, axes = plt.subplots(
    nrows=top_n,
    ncols=2,
    figsize=(16, 5 * top_n),
    squeeze=False
)
```

```
for indice, trial in enumerate(melhores_trials):
    tf.keras.backend.clear_session()
    tf.keras.utils.set_random_seed(random_state)
```

```
    trial_id = str(trial.trial_id).zfill(2)
    hp = trial.hyperparameters
```

```
    model = build_model(hp)
```

```
    early_stopping = tf.keras.callbacks.EarlyStopping(
        monitor="val_accuracy",
        mode="max",
        patience=patience,
        restore_best_weights=True
    )
```

```
    history = model.fit(
        X_train,
        y_train,
        validation_data=(X_val, y_val),
        epochs=epochs,
        batch_size=batch_size,
        callbacks=[early_stopping],
        shuffle=True,
        verbose=1
    )
```

```
    train_accuracy = history.history["accuracy"]
    val_accuracy = history.history["val_accuracy"]
    train_loss = history.history["loss"]
```

```

val_loss = history.history["val_loss"]

epocas = range(1, len(val_accuracy) + 1)

melhor_epoca = int(tf.argmax(val_accuracy).numpy()) + 1
melhor_val_accuracy = val_accuracy[melhor_epoca - 1]

dense_units = hp.get("dense_units")
dropout_rate = hp.get("dropout_rate")
learning_rate = hp.get("learning_rate")

parametros = (
    f"Trial {trial_id} | "
    f"dense={dense_units} | "
    f"dropout={dropout_rate:.1f} | "
    f"lr={learning_rate:.0e}"
)

ax_accuracy = axes[indice, 0]
ax_loss = axes[indice, 1]

# Gráfico de acurácia
ax_accuracy.plot(
    epocas,
    train_accuracy,
    label="Treino"
)

ax_accuracy.plot(
    epocas,
    val_accuracy,
    label="Validação"
)

ax_accuracy.axvline(
    melhor_epoca,
    linestyle="--",
    linewidth=1.5,
    alpha=0.8,
    label=f"Melhor época: {melhor_epoca}"
)

ax_accuracy.scatter(
    melhor_epoca,
    melhor_val_accuracy,
    zorder=3
)

```



```

ax_accuracy.set_title(
    f"{parametros}\n"
    f"Evolução da acurácia | "
    f"melhor val_accuracy={melhor_val_accuracy:.4f}"
)
ax_accuracy.set_xlabel("Época")
ax_accuracy.set_ylabel("Acurácia")
ax_accuracy.set_xlim(1, epochs)
ax_accuracy.grid(True, alpha=0.3)
ax_accuracy.legend()

# Gráfico de loss
ax_loss.plot(
    epocas,
    train_loss,
    label="Treino"
)

ax_loss.plot(
    epocas,
    val_loss,
    label="Validação"
)

ax_loss.axvline(
    melhor_epoca,
    linestyle="--",
    linewidth=1.5,
    alpha=0.8,
    label=f"Melhor época: {melhor_epoca}"
)

ax_loss.set_title(
    f"{parametros}\n"
    f"Evolução da loss"
)
ax_loss.set_xlabel("Época")
ax_loss.set_ylabel("Loss")
ax_loss.set_xlim(1, epochs)
ax_loss.grid(True, alpha=0.3)
ax_loss.legend()

resultados_retreino[trial_id] = {
    "modelo": model,
    "historico": history,
    "hiperparametros": {

```

```

        "dense_units": dense_units,
        "dropout_rate": dropout_rate,
        "learning_rate": learning_rate
    },
    "melhor_epoca": melhor_epoca,
    "melhor_val_accuracy": melhor_val_accuracy
}

plt.tight_layout()
plt.show()

return resultados_retreino

```

```

[ ]: resultados_retreino_topN = retrainar_e_plotar_melhores_modelos(
    tuner=tuner,
    build_model=build_model,
    X_train=X_tune_train_array,
    y_train=y_tune_train_array,
    X_val=X_val_array,
    y_val=y_val_array,
    top_n=6,
    epochs=50,
    batch_size=64,
    patience=3,
    random_state=RANDOM_STATE
)

```

```

Epoch 1/50
633/633          11s 17ms/step -
accuracy: 0.5784 - loss: 0.6906 - val_accuracy: 0.6402 - val_loss: 0.6866
Epoch 2/50
633/633          11s 17ms/step -
accuracy: 0.6661 - loss: 0.6787 - val_accuracy: 0.7227 - val_loss: 0.6684
Epoch 3/50
633/633          11s 17ms/step -
accuracy: 0.7180 - loss: 0.6540 - val_accuracy: 0.7764 - val_loss: 0.6361
Epoch 4/50
633/633          11s 17ms/step -
accuracy: 0.7581 - loss: 0.6174 - val_accuracy: 0.8082 - val_loss: 0.5934
Epoch 5/50
633/633          11s 17ms/step -
accuracy: 0.7799 - loss: 0.5753 - val_accuracy: 0.8256 - val_loss: 0.5476
Epoch 6/50
633/633          11s 17ms/step -
accuracy: 0.7972 - loss: 0.5328 - val_accuracy: 0.8382 - val_loss: 0.5043
Epoch 7/50
633/633          11s 17ms/step -
accuracy: 0.8117 - loss: 0.4951 - val_accuracy: 0.8491 - val_loss: 0.4662

```

Epoch 8/50  
633/633 11s 17ms/step -  
accuracy: 0.8240 - loss: 0.4634 - val\_accuracy: 0.8564 - val\_loss: 0.4335  
Epoch 9/50  
633/633 11s 17ms/step -  
accuracy: 0.8338 - loss: 0.4346 - val\_accuracy: 0.8620 - val\_loss: 0.4059  
Epoch 10/50  
633/633 11s 17ms/step -  
accuracy: 0.8436 - loss: 0.4112 - val\_accuracy: 0.8693 - val\_loss: 0.3829  
Epoch 11/50  
633/633 11s 17ms/step -  
accuracy: 0.8500 - loss: 0.3899 - val\_accuracy: 0.8716 - val\_loss: 0.3636  
Epoch 12/50  
633/633 11s 17ms/step -  
accuracy: 0.8561 - loss: 0.3721 - val\_accuracy: 0.8751 - val\_loss: 0.3472  
Epoch 13/50  
633/633 11s 17ms/step -  
accuracy: 0.8640 - loss: 0.3542 - val\_accuracy: 0.8762 - val\_loss: 0.3335  
Epoch 14/50  
633/633 11s 17ms/step -  
accuracy: 0.8682 - loss: 0.3418 - val\_accuracy: 0.8784 - val\_loss: 0.3216  
Epoch 15/50  
633/633 11s 17ms/step -  
accuracy: 0.8763 - loss: 0.3275 - val\_accuracy: 0.8809 - val\_loss: 0.3116  
Epoch 16/50  
633/633 11s 17ms/step -  
accuracy: 0.8792 - loss: 0.3166 - val\_accuracy: 0.8818 - val\_loss: 0.3031  
Epoch 17/50  
633/633 11s 17ms/step -  
accuracy: 0.8828 - loss: 0.3071 - val\_accuracy: 0.8842 - val\_loss: 0.2957  
Epoch 18/50  
633/633 11s 17ms/step -  
accuracy: 0.8879 - loss: 0.2971 - val\_accuracy: 0.8860 - val\_loss: 0.2894  
Epoch 19/50  
633/633 11s 17ms/step -  
accuracy: 0.8906 - loss: 0.2917 - val\_accuracy: 0.8862 - val\_loss: 0.2840  
Epoch 20/50  
633/633 11s 17ms/step -  
accuracy: 0.8949 - loss: 0.2814 - val\_accuracy: 0.8891 - val\_loss: 0.2792  
Epoch 21/50  
633/633 11s 17ms/step -  
accuracy: 0.8968 - loss: 0.2759 - val\_accuracy: 0.8902 - val\_loss: 0.2750  
Epoch 22/50  
633/633 11s 17ms/step -  
accuracy: 0.9014 - loss: 0.2683 - val\_accuracy: 0.8907 - val\_loss: 0.2714  
Epoch 23/50  
633/633 11s 17ms/step -  
accuracy: 0.9033 - loss: 0.2621 - val\_accuracy: 0.8918 - val\_loss: 0.2683

Epoch 24/50  
633/633 11s 17ms/step -  
accuracy: 0.9057 - loss: 0.2582 - val\_accuracy: 0.8927 - val\_loss: 0.2654

Epoch 25/50  
633/633 11s 17ms/step -  
accuracy: 0.9083 - loss: 0.2509 - val\_accuracy: 0.8929 - val\_loss: 0.2632

Epoch 26/50  
633/633 11s 17ms/step -  
accuracy: 0.9095 - loss: 0.2456 - val\_accuracy: 0.8951 - val\_loss: 0.2609

Epoch 27/50  
633/633 11s 17ms/step -  
accuracy: 0.9133 - loss: 0.2409 - val\_accuracy: 0.8947 - val\_loss: 0.2590

Epoch 28/50  
633/633 11s 17ms/step -  
accuracy: 0.9143 - loss: 0.2369 - val\_accuracy: 0.8944 - val\_loss: 0.2575

Epoch 29/50  
633/633 11s 18ms/step -  
accuracy: 0.9183 - loss: 0.2313 - val\_accuracy: 0.8962 - val\_loss: 0.2561

Epoch 30/50  
633/633 11s 17ms/step -  
accuracy: 0.9181 - loss: 0.2295 - val\_accuracy: 0.8960 - val\_loss: 0.2547

Epoch 31/50  
633/633 11s 17ms/step -  
accuracy: 0.9198 - loss: 0.2239 - val\_accuracy: 0.8976 - val\_loss: 0.2539

Epoch 32/50  
633/633 11s 17ms/step -  
accuracy: 0.9242 - loss: 0.2200 - val\_accuracy: 0.8973 - val\_loss: 0.2529

Epoch 33/50  
633/633 11s 17ms/step -  
accuracy: 0.9245 - loss: 0.2162 - val\_accuracy: 0.9000 - val\_loss: 0.2524

Epoch 34/50  
633/633 11s 17ms/step -  
accuracy: 0.9250 - loss: 0.2129 - val\_accuracy: 0.9004 - val\_loss: 0.2518

Epoch 35/50  
633/633 11s 17ms/step -  
accuracy: 0.9271 - loss: 0.2102 - val\_accuracy: 0.8998 - val\_loss: 0.2514

Epoch 36/50  
633/633 11s 17ms/step -  
accuracy: 0.9279 - loss: 0.2077 - val\_accuracy: 0.9002 - val\_loss: 0.2510

Epoch 37/50  
633/633 11s 17ms/step -  
accuracy: 0.9298 - loss: 0.2040 - val\_accuracy: 0.8998 - val\_loss: 0.2507

Epoch 1/50  
633/633 12s 18ms/step -  
accuracy: 0.7025 - loss: 0.6768 - val\_accuracy: 0.7693 - val\_loss: 0.6410

Epoch 2/50  
633/633 11s 17ms/step -  
accuracy: 0.8034 - loss: 0.5731 - val\_accuracy: 0.8322 - val\_loss: 0.4996

Epoch 3/50  
633/633 11s 17ms/step -  
accuracy: 0.8420 - loss: 0.4452 - val\_accuracy: 0.8591 - val\_loss: 0.3963  
Epoch 4/50  
633/633 11s 18ms/step -  
accuracy: 0.8645 - loss: 0.3663 - val\_accuracy: 0.8711 - val\_loss: 0.3409  
Epoch 5/50  
633/633 11s 17ms/step -  
accuracy: 0.8792 - loss: 0.3206 - val\_accuracy: 0.8762 - val\_loss: 0.3095  
Epoch 6/50  
633/633 11s 17ms/step -  
accuracy: 0.8905 - loss: 0.2895 - val\_accuracy: 0.8829 - val\_loss: 0.2901  
Epoch 7/50  
633/633 11s 18ms/step -  
accuracy: 0.8986 - loss: 0.2685 - val\_accuracy: 0.8876 - val\_loss: 0.2773  
Epoch 8/50  
633/633 11s 17ms/step -  
accuracy: 0.9053 - loss: 0.2510 - val\_accuracy: 0.8918 - val\_loss: 0.2681  
Epoch 9/50  
633/633 11s 18ms/step -  
accuracy: 0.9120 - loss: 0.2366 - val\_accuracy: 0.8938 - val\_loss: 0.2619  
Epoch 10/50  
633/633 11s 17ms/step -  
accuracy: 0.9167 - loss: 0.2250 - val\_accuracy: 0.8953 - val\_loss: 0.2579  
Epoch 11/50  
633/633 11s 17ms/step -  
accuracy: 0.9213 - loss: 0.2147 - val\_accuracy: 0.8967 - val\_loss: 0.2548  
Epoch 12/50  
633/633 11s 17ms/step -  
accuracy: 0.9239 - loss: 0.2056 - val\_accuracy: 0.8991 - val\_loss: 0.2530  
Epoch 13/50  
633/633 11s 17ms/step -  
accuracy: 0.9288 - loss: 0.1957 - val\_accuracy: 0.9000 - val\_loss: 0.2521  
Epoch 14/50  
633/633 11s 17ms/step -  
accuracy: 0.9318 - loss: 0.1893 - val\_accuracy: 0.8998 - val\_loss: 0.2516  
Epoch 15/50  
633/633 11s 17ms/step -  
accuracy: 0.9348 - loss: 0.1819 - val\_accuracy: 0.9004 - val\_loss: 0.2516  
Epoch 16/50  
633/633 11s 17ms/step -  
accuracy: 0.9377 - loss: 0.1765 - val\_accuracy: 0.9002 - val\_loss: 0.2519  
Epoch 17/50  
633/633 11s 17ms/step -  
accuracy: 0.9412 - loss: 0.1692 - val\_accuracy: 0.9016 - val\_loss: 0.2541  
Epoch 18/50  
633/633 11s 17ms/step -  
accuracy: 0.9429 - loss: 0.1625 - val\_accuracy: 0.9002 - val\_loss: 0.2550

Epoch 19/50  
633/633 11s 17ms/step -  
accuracy: 0.9457 - loss: 0.1583 - val\_accuracy: 0.8991 - val\_loss: 0.2564  
Epoch 20/50  
633/633 11s 17ms/step -  
accuracy: 0.9472 - loss: 0.1534 - val\_accuracy: 0.8998 - val\_loss: 0.2583  
Epoch 1/50  
633/633 12s 17ms/step -  
accuracy: 0.6210 - loss: 0.6849 - val\_accuracy: 0.6951 - val\_loss: 0.6683  
Epoch 2/50  
633/633 11s 17ms/step -  
accuracy: 0.7393 - loss: 0.6305 - val\_accuracy: 0.8044 - val\_loss: 0.5810  
Epoch 3/50  
633/633 11s 17ms/step -  
accuracy: 0.7967 - loss: 0.5364 - val\_accuracy: 0.8424 - val\_loss: 0.4802  
Epoch 4/50  
633/633 10s 16ms/step -  
accuracy: 0.8281 - loss: 0.4554 - val\_accuracy: 0.8631 - val\_loss: 0.4082  
Epoch 5/50  
633/633 10s 16ms/step -  
accuracy: 0.8478 - loss: 0.3981 - val\_accuracy: 0.8702 - val\_loss: 0.3614  
Epoch 6/50  
633/633 11s 17ms/step -  
accuracy: 0.8619 - loss: 0.3584 - val\_accuracy: 0.8753 - val\_loss: 0.3304  
Epoch 7/50  
633/633 10s 17ms/step -  
accuracy: 0.8741 - loss: 0.3297 - val\_accuracy: 0.8798 - val\_loss: 0.3093  
Epoch 8/50  
633/633 10s 16ms/step -  
accuracy: 0.8833 - loss: 0.3080 - val\_accuracy: 0.8836 - val\_loss: 0.2938  
Epoch 9/50  
633/633 10s 16ms/step -  
accuracy: 0.8911 - loss: 0.2891 - val\_accuracy: 0.8878 - val\_loss: 0.2821  
Epoch 10/50  
633/633 10s 16ms/step -  
accuracy: 0.8987 - loss: 0.2744 - val\_accuracy: 0.8904 - val\_loss: 0.2731  
Epoch 11/50  
633/633 10s 16ms/step -  
accuracy: 0.9039 - loss: 0.2607 - val\_accuracy: 0.8922 - val\_loss: 0.2669  
Epoch 12/50  
633/633 10s 16ms/step -  
accuracy: 0.9082 - loss: 0.2501 - val\_accuracy: 0.8944 - val\_loss: 0.2621  
Epoch 13/50  
633/633 10s 16ms/step -  
accuracy: 0.9124 - loss: 0.2386 - val\_accuracy: 0.8958 - val\_loss: 0.2586  
Epoch 14/50  
633/633 10s 16ms/step -  
accuracy: 0.9169 - loss: 0.2307 - val\_accuracy: 0.8973 - val\_loss: 0.2561

Epoch 15/50  
633/633 10s 17ms/step -  
accuracy: 0.9208 - loss: 0.2214 - val\_accuracy: 0.8984 - val\_loss: 0.2543

Epoch 16/50  
633/633 10s 16ms/step -  
accuracy: 0.9235 - loss: 0.2150 - val\_accuracy: 0.8980 - val\_loss: 0.2535

Epoch 17/50  
633/633 10s 16ms/step -  
accuracy: 0.9268 - loss: 0.2079 - val\_accuracy: 0.9007 - val\_loss: 0.2513

Epoch 18/50  
633/633 10s 16ms/step -  
accuracy: 0.9305 - loss: 0.2011 - val\_accuracy: 0.8998 - val\_loss: 0.2517

Epoch 19/50  
633/633 10s 16ms/step -  
accuracy: 0.9318 - loss: 0.1974 - val\_accuracy: 0.8996 - val\_loss: 0.2514

Epoch 20/50  
633/633 10s 16ms/step -  
accuracy: 0.9361 - loss: 0.1891 - val\_accuracy: 0.9004 - val\_loss: 0.2520

Epoch 1/50  
633/633 11s 16ms/step -  
accuracy: 0.6653 - loss: 0.6876 - val\_accuracy: 0.7227 - val\_loss: 0.6758

Epoch 2/50  
633/633 10s 16ms/step -  
accuracy: 0.7632 - loss: 0.6470 - val\_accuracy: 0.7869 - val\_loss: 0.6105

Epoch 3/50  
633/633 10s 16ms/step -  
accuracy: 0.8097 - loss: 0.5655 - val\_accuracy: 0.8251 - val\_loss: 0.5220

Epoch 4/50  
633/633 11s 17ms/step -  
accuracy: 0.8371 - loss: 0.4813 - val\_accuracy: 0.8438 - val\_loss: 0.4470

Epoch 5/50  
633/633 11s 18ms/step -  
accuracy: 0.8534 - loss: 0.4168 - val\_accuracy: 0.8593 - val\_loss: 0.3940

Epoch 6/50  
633/633 11s 17ms/step -  
accuracy: 0.8666 - loss: 0.3711 - val\_accuracy: 0.8671 - val\_loss: 0.3575

Epoch 7/50  
633/633 11s 17ms/step -  
accuracy: 0.8755 - loss: 0.3379 - val\_accuracy: 0.8731 - val\_loss: 0.3320

Epoch 8/50  
633/633 11s 18ms/step -  
accuracy: 0.8819 - loss: 0.3135 - val\_accuracy: 0.8773 - val\_loss: 0.3133

Epoch 9/50  
633/633 11s 17ms/step -  
accuracy: 0.8892 - loss: 0.2942 - val\_accuracy: 0.8789 - val\_loss: 0.2996

Epoch 10/50  
633/633 11s 17ms/step -  
accuracy: 0.8957 - loss: 0.2784 - val\_accuracy: 0.8811 - val\_loss: 0.2891

Epoch 11/50  
633/633 11s 17ms/step -  
accuracy: 0.8993 - loss: 0.2655 - val\_accuracy: 0.8833 - val\_loss: 0.2808

Epoch 12/50  
633/633 11s 17ms/step -  
accuracy: 0.9046 - loss: 0.2540 - val\_accuracy: 0.8871 - val\_loss: 0.2744

Epoch 13/50  
633/633 11s 17ms/step -  
accuracy: 0.9082 - loss: 0.2445 - val\_accuracy: 0.8891 - val\_loss: 0.2692

Epoch 14/50  
633/633 11s 17ms/step -  
accuracy: 0.9115 - loss: 0.2356 - val\_accuracy: 0.8931 - val\_loss: 0.2651

Epoch 15/50  
633/633 11s 17ms/step -  
accuracy: 0.9140 - loss: 0.2278 - val\_accuracy: 0.8938 - val\_loss: 0.2617

Epoch 16/50  
633/633 12s 18ms/step -  
accuracy: 0.9177 - loss: 0.2207 - val\_accuracy: 0.8956 - val\_loss: 0.2590

Epoch 17/50  
633/633 11s 17ms/step -  
accuracy: 0.9206 - loss: 0.2138 - val\_accuracy: 0.8969 - val\_loss: 0.2569

Epoch 18/50  
633/633 11s 17ms/step -  
accuracy: 0.9230 - loss: 0.2082 - val\_accuracy: 0.8973 - val\_loss: 0.2552

Epoch 19/50  
633/633 11s 17ms/step -  
accuracy: 0.9256 - loss: 0.2023 - val\_accuracy: 0.8989 - val\_loss: 0.2539

Epoch 20/50  
633/633 11s 17ms/step -  
accuracy: 0.9278 - loss: 0.1975 - val\_accuracy: 0.8991 - val\_loss: 0.2529

Epoch 21/50  
633/633 11s 17ms/step -  
accuracy: 0.9290 - loss: 0.1922 - val\_accuracy: 0.8996 - val\_loss: 0.2525

Epoch 22/50  
633/633 11s 17ms/step -  
accuracy: 0.9319 - loss: 0.1874 - val\_accuracy: 0.9002 - val\_loss: 0.2519

Epoch 23/50  
633/633 11s 17ms/step -  
accuracy: 0.9336 - loss: 0.1823 - val\_accuracy: 0.9016 - val\_loss: 0.2519

Epoch 24/50  
633/633 11s 17ms/step -  
accuracy: 0.9352 - loss: 0.1785 - val\_accuracy: 0.9024 - val\_loss: 0.2520

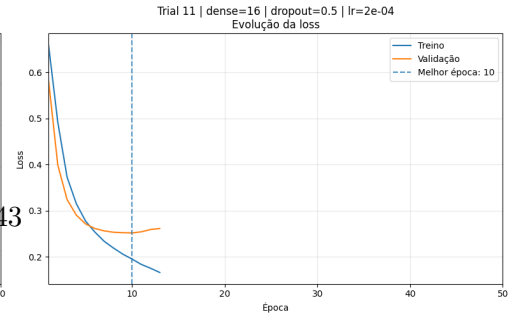
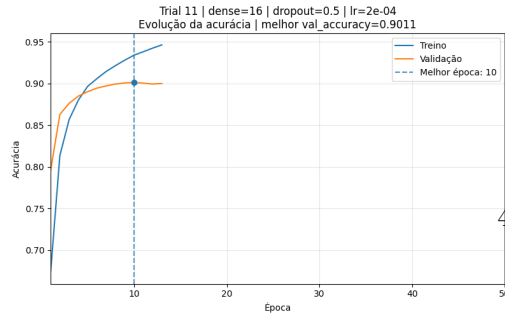
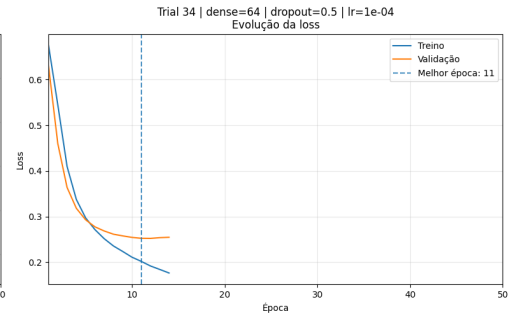
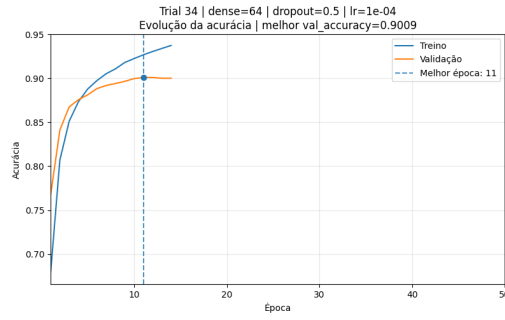
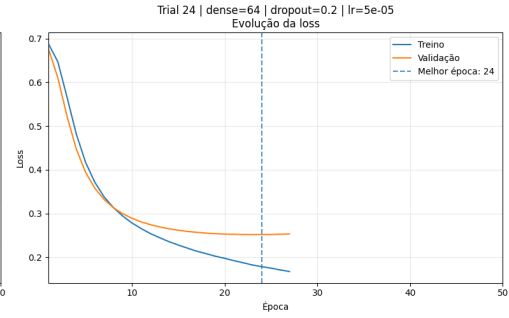
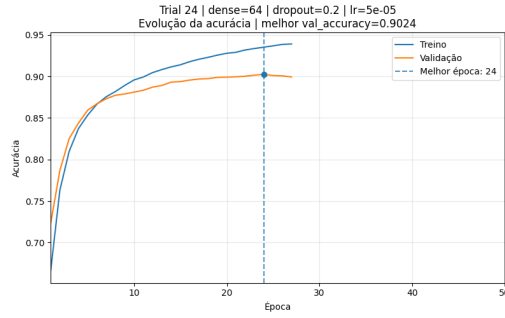
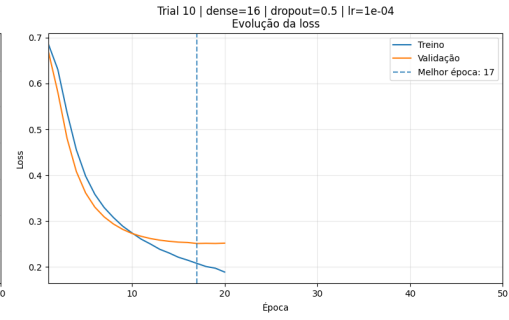
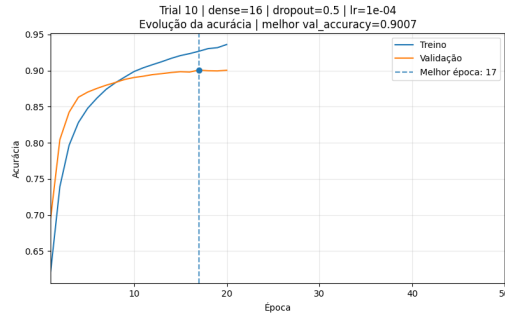
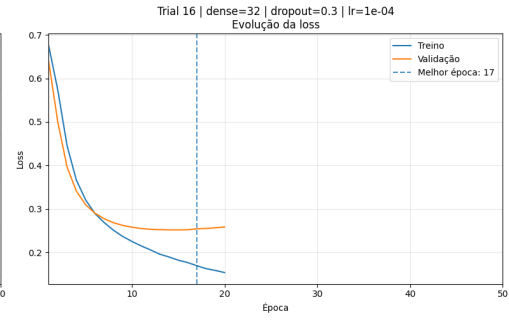
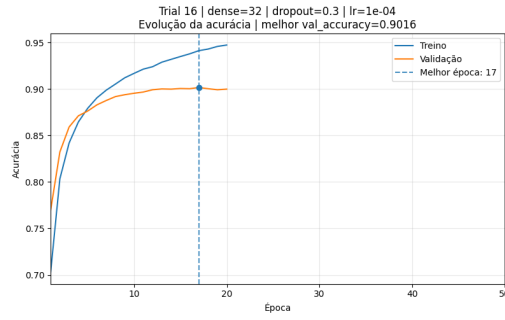
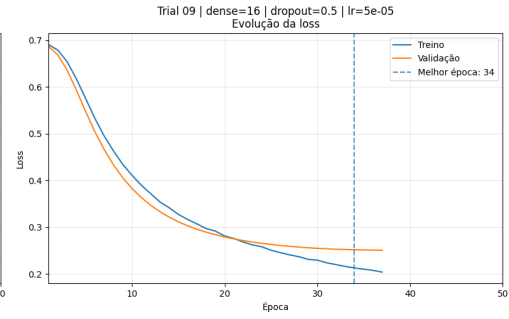
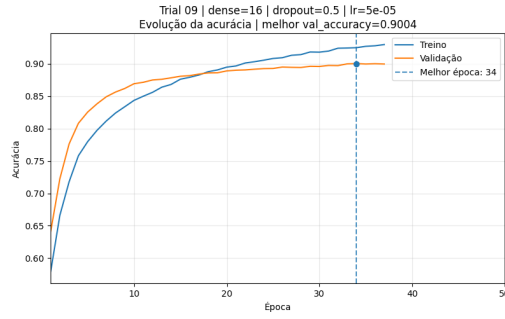
Epoch 25/50  
633/633 11s 17ms/step -  
accuracy: 0.9368 - loss: 0.1751 - val\_accuracy: 0.9011 - val\_loss: 0.2521

Epoch 26/50  
633/633 11s 17ms/step -  
accuracy: 0.9385 - loss: 0.1710 - val\_accuracy: 0.9007 - val\_loss: 0.2525



Epoch 27/50  
633/633 11s 17ms/step -  
accuracy: 0.9390 - loss: 0.1674 - val\_accuracy: 0.8993 - val\_loss: 0.2533  
Epoch 1/50  
633/633 12s 18ms/step -  
accuracy: 0.6786 - loss: 0.6750 - val\_accuracy: 0.7671 - val\_loss: 0.6298  
Epoch 2/50  
633/633 11s 17ms/step -  
accuracy: 0.8070 - loss: 0.5447 - val\_accuracy: 0.8411 - val\_loss: 0.4598  
Epoch 3/50  
633/633 11s 17ms/step -  
accuracy: 0.8509 - loss: 0.4098 - val\_accuracy: 0.8673 - val\_loss: 0.3639  
Epoch 4/50  
633/633 11s 17ms/step -  
accuracy: 0.8733 - loss: 0.3375 - val\_accuracy: 0.8756 - val\_loss: 0.3182  
Epoch 5/50  
633/633 11s 17ms/step -  
accuracy: 0.8877 - loss: 0.2972 - val\_accuracy: 0.8811 - val\_loss: 0.2930  
Epoch 6/50  
633/633 11s 17ms/step -  
accuracy: 0.8974 - loss: 0.2716 - val\_accuracy: 0.8882 - val\_loss: 0.2776  
Epoch 7/50  
633/633 11s 17ms/step -  
accuracy: 0.9051 - loss: 0.2517 - val\_accuracy: 0.8918 - val\_loss: 0.2685  
Epoch 8/50  
633/633 11s 17ms/step -  
accuracy: 0.9107 - loss: 0.2357 - val\_accuracy: 0.8940 - val\_loss: 0.2614  
Epoch 9/50  
633/633 11s 17ms/step -  
accuracy: 0.9179 - loss: 0.2236 - val\_accuracy: 0.8964 - val\_loss: 0.2577  
Epoch 10/50  
633/633 11s 17ms/step -  
accuracy: 0.9223 - loss: 0.2111 - val\_accuracy: 0.8996 - val\_loss: 0.2544  
Epoch 11/50  
633/633 11s 17ms/step -  
accuracy: 0.9266 - loss: 0.2019 - val\_accuracy: 0.9009 - val\_loss: 0.2524  
Epoch 12/50  
633/633 11s 17ms/step -  
accuracy: 0.9303 - loss: 0.1918 - val\_accuracy: 0.9009 - val\_loss: 0.2522  
Epoch 13/50  
633/633 11s 17ms/step -  
accuracy: 0.9338 - loss: 0.1842 - val\_accuracy: 0.9000 - val\_loss: 0.2538  
Epoch 14/50  
633/633 11s 17ms/step -  
accuracy: 0.9373 - loss: 0.1764 - val\_accuracy: 0.9000 - val\_loss: 0.2547  
Epoch 1/50  
633/633 12s 18ms/step -  
accuracy: 0.6733 - loss: 0.6598 - val\_accuracy: 0.7956 - val\_loss: 0.5828

Epoch 2/50  
633/633 11s 17ms/step -  
accuracy: 0.8136 - loss: 0.4918 - val\_accuracy: 0.8629 - val\_loss: 0.3990  
Epoch 3/50  
633/633 11s 17ms/step -  
accuracy: 0.8567 - loss: 0.3730 - val\_accuracy: 0.8762 - val\_loss: 0.3243  
Epoch 4/50  
633/633 11s 17ms/step -  
accuracy: 0.8801 - loss: 0.3152 - val\_accuracy: 0.8849 - val\_loss: 0.2905  
Epoch 5/50  
633/633 11s 17ms/step -  
accuracy: 0.8966 - loss: 0.2775 - val\_accuracy: 0.8902 - val\_loss: 0.2714  
Epoch 6/50  
633/633 11s 17ms/step -  
accuracy: 0.9058 - loss: 0.2537 - val\_accuracy: 0.8944 - val\_loss: 0.2614  
Epoch 7/50  
633/633 11s 17ms/step -  
accuracy: 0.9144 - loss: 0.2337 - val\_accuracy: 0.8971 - val\_loss: 0.2563  
Epoch 8/50  
633/633 11s 17ms/step -  
accuracy: 0.9214 - loss: 0.2192 - val\_accuracy: 0.8993 - val\_loss: 0.2534  
Epoch 9/50  
633/633 11s 17ms/step -  
accuracy: 0.9280 - loss: 0.2060 - val\_accuracy: 0.9007 - val\_loss: 0.2524  
Epoch 10/50  
633/633 11s 17ms/step -  
accuracy: 0.9340 - loss: 0.1953 - val\_accuracy: 0.9011 - val\_loss: 0.2519  
Epoch 11/50  
633/633 11s 17ms/step -  
accuracy: 0.9381 - loss: 0.1837 - val\_accuracy: 0.9004 - val\_loss: 0.2543  
Epoch 12/50  
633/633 11s 17ms/step -  
accuracy: 0.9423 - loss: 0.1753 - val\_accuracy: 0.8993 - val\_loss: 0.2593  
Epoch 13/50  
633/633 11s 17ms/step -  
accuracy: 0.9463 - loss: 0.1661 - val\_accuracy: 0.9000 - val\_loss: 0.2615



Embora algumas configurações tenham apresentado acurácia de validação ligeiramente superior, de modo geral as diferenças observadas foram pequenas. Assim, vamos considerar na escolha do modelo final não apenas o melhor valor pontual de acurácia, mas também a estabilidade do treinamento, a complexidade da arquitetura e os sinais de overfitting.

Escolhemos o trial 09, modelo com `dense_units=16`, `dropout_rate=0,5` e `learning_rate=5e-5`. Essa configuração apresentou convergência mais gradual, menor diferença entre as curvas de treino e validação e comportamento mais estável da loss de validação. Além disso, utiliza uma camada densa menor e maior regularização por dropout, reduzindo a complexidade do modelo sem perda relevante de desempenho.

Portanto, a escolha prioriza um equilíbrio entre acurácia, capacidade de generalização e simplicidade arquitetural.

#### 1.1.10 5º Passo:

- Quando definida uma boa arquitetura neural, faça o **treinamento final** do modelo com todos os dados de treinamento (sem validação).

```
[54]: # =====  
# Configuração do modelo final  
# =====  
  
FINAL_EMBED_DIM = 50  
FINAL_DENSE_UNITS = 16  
FINAL_DROPOUT_RATE = 0.5  
FINAL_LEARNING_RATE = 5e-5  
FINAL_BATCH_SIZE = 64  
  
# Melhor época observada para a configuração escolhida  
FINAL_EPOCHS = 34  
  
# =====  
# Conversão dos dados completos de treino  
# =====  
  
X_train_final = tf.convert_to_tensor(  
    X_train.astype(str).to_numpy().reshape(-1, 1),  
    dtype=tf.string  
)  
  
y_train_final = tf.convert_to_tensor(  
    y_train.to_numpy(),  
    dtype=tf.float32  
)
```

```
print(f"Formato de X_train_final: {X_train_final.shape}")
print(f"Formato de y_train_final: {y_train_final.shape}")
```

Formato de X\_train\_final: (45000, 1)

Formato de y\_train\_final: (45000,)

```
[55]: # =====
# Vetorização final
# =====

vectorizer_final = layers.TextVectorization(
    standardize=Preprocessor,
    max_tokens=VOCAB_SIZE,
    output_mode="int",
    output_sequence_length=MAX_LENGTH
)

vectorizer_final.adapt(X_train_final)

print(
    f"Tamanho efetivo do vocabulário: "
    f"{vectorizer_final.vocabulary_size()}"
)
```

Tamanho efetivo do vocabulário: 15000

```
[56]: # =====
# Construção do modelo final
# =====

def criar_modelo_final():
    x_in = layers.Input(
        shape=(1,),
        dtype=tf.string,
        name="review"
    )

    x = vectorizer_final(x_in)

    x = layers.Embedding(
        input_dim=vectorizer_final.vocabulary_size(),
        output_dim=FINAL_EMBED_DIM,
        mask_zero=True,
        name="embedding"
    )(x)

    x = layers.GlobalAveragePooling1D(
```

```

        name="global_average_pooling"
    )(x)

    x = layers.Dense(
        units=FINAL_DENSE_UNITS,
        activation="relu",
        name="hidden"
    )(x)

    x = layers.Dropout(
        rate=FINAL_DROPOUT_RATE,
        name="dropout"
    )(x)

    y_out = layers.Dense(
        units=1,
        activation="sigmoid",
        name="output"
    )(x)

    model = tf.keras.Model(
        inputs=x_in,
        outputs=y_out,
        name="imdb_sentiment_classifier"
    )

    optimizer = tf.keras.optimizers.Adam(
        learning_rate=FINAL_LEARNING_RATE
    )

    model.compile(
        optimizer=optimizer,
        loss="binary_crossentropy",
        metrics=[
            tf.keras.metrics.BinaryAccuracy(
                name="accuracy"
            )
        ]
    )

    return model

```

```

[57]: # =====
# Treinamento final
# =====

tf.keras.backend.clear_session()

```

```
tf.keras.utils.set_random_seed(RANDOM_STATE)

model_final = criar_modelo_final()

model_final.summary()
```

Model: "imdb\_sentiment\_classifier"

| Layer (type)                                                      | Output Shape    | Param # | Connected to                        |
|-------------------------------------------------------------------|-----------------|---------|-------------------------------------|
| review ( <a href="#">InputLayer</a> )                             | (None, 1)       | 0       | -                                   |
| text_vectorization<br>( <a href="#">TextVectorization</a> )       | (None, 581)     | 0       | review[0][0]                        |
| embedding<br>( <a href="#">Embedding</a> )                        | (None, 581, 50) | 750,000 | text_vectorizati...                 |
| not_equal<br>( <a href="#">NotEqual</a> )                         | (None, 581)     | 0       | text_vectorizati...                 |
| global_average_poo...<br>( <a href="#">GlobalAveragePool...</a> ) | (None, 50)      | 0       | embedding[0][0],<br>not_equal[0][0] |
| hidden ( <a href="#">Dense</a> )                                  | (None, 16)      | 816     | global_average_p...                 |
| dropout ( <a href="#">Dropout</a> )                               | (None, 16)      | 0       | hidden[0][0]                        |
| output ( <a href="#">Dense</a> )                                  | (None, 1)       | 17      | dropout[0][0]                       |

Total params: 750,833 (2.86 MB)

Trainable params: 750,833 (2.86 MB)

Non-trainable params: 0 (0.00 B)

```
[58]: history_final = model_final.fit(
        X_train_final,
        y_train_final,
        epochs=FINAL_EPOCHS,
        batch_size=FINAL_BATCH_SIZE,
        shuffle=True,
```

```
    verbose=1
)
```

```
Epoch 1/34
704/704      13s 17ms/step -
accuracy: 0.5674 - loss: 0.6903
Epoch 2/34
704/704      11s 16ms/step -
accuracy: 0.6553 - loss: 0.6760
Epoch 3/34
704/704      11s 16ms/step -
accuracy: 0.7231 - loss: 0.6452
Epoch 4/34
704/704      11s 16ms/step -
accuracy: 0.7668 - loss: 0.5992
Epoch 5/34
704/704      11s 16ms/step -
accuracy: 0.7914 - loss: 0.5515
Epoch 6/34
704/704      11s 16ms/step -
accuracy: 0.8097 - loss: 0.5055
Epoch 7/34
704/704      11s 16ms/step -
accuracy: 0.8217 - loss: 0.4658
Epoch 8/34
704/704      11s 16ms/step -
accuracy: 0.8336 - loss: 0.4336
Epoch 9/34
704/704      11s 16ms/step -
accuracy: 0.8448 - loss: 0.4067
Epoch 10/34
704/704      11s 16ms/step -
accuracy: 0.8529 - loss: 0.3826
Epoch 11/34
704/704      11s 16ms/step -
accuracy: 0.8593 - loss: 0.3629
Epoch 12/34
704/704      11s 16ms/step -
accuracy: 0.8685 - loss: 0.3452
Epoch 13/34
704/704      11s 16ms/step -
accuracy: 0.8747 - loss: 0.3303
Epoch 14/34
704/704      11s 16ms/step -
accuracy: 0.8779 - loss: 0.3178
Epoch 15/34
704/704      11s 16ms/step -
accuracy: 0.8820 - loss: 0.3059
```



Epoch 16/34  
704/704 11s 16ms/step -  
accuracy: 0.8873 - loss: 0.2952  
Epoch 17/34  
704/704 11s 16ms/step -  
accuracy: 0.8906 - loss: 0.2877  
Epoch 18/34  
704/704 11s 16ms/step -  
accuracy: 0.8947 - loss: 0.2798  
Epoch 19/34  
704/704 11s 16ms/step -  
accuracy: 0.8973 - loss: 0.2709  
Epoch 20/34  
704/704 11s 16ms/step -  
accuracy: 0.9022 - loss: 0.2634  
Epoch 21/34  
704/704 11s 16ms/step -  
accuracy: 0.9037 - loss: 0.2591  
Epoch 22/34  
704/704 11s 16ms/step -  
accuracy: 0.9077 - loss: 0.2525  
Epoch 23/34  
704/704 11s 16ms/step -  
accuracy: 0.9074 - loss: 0.2466  
Epoch 24/34  
704/704 11s 16ms/step -  
accuracy: 0.9124 - loss: 0.2407  
Epoch 25/34  
704/704 11s 16ms/step -  
accuracy: 0.9152 - loss: 0.2366  
Epoch 26/34  
704/704 11s 16ms/step -  
accuracy: 0.9152 - loss: 0.2324  
Epoch 27/34  
704/704 11s 16ms/step -  
accuracy: 0.9179 - loss: 0.2270  
Epoch 28/34  
704/704 11s 16ms/step -  
accuracy: 0.9195 - loss: 0.2228  
Epoch 29/34  
704/704 11s 16ms/step -  
accuracy: 0.9209 - loss: 0.2194  
Epoch 30/34  
704/704 11s 16ms/step -  
accuracy: 0.9230 - loss: 0.2152  
Epoch 31/34  
704/704 12s 16ms/step -  
accuracy: 0.9246 - loss: 0.2139

```
Epoch 32/34
704/704          11s 16ms/step -
accuracy: 0.9273 - loss: 0.2087
Epoch 33/34
704/704          12s 17ms/step -
accuracy: 0.9281 - loss: 0.2042
Epoch 34/34
704/704          11s 16ms/step -
accuracy: 0.9294 - loss: 0.2030
```

```
[61]: import matplotlib.pyplot as plt

def plotar_historico_modelo_final(history):
    """
    Plota a evolução da acurácia e da loss do modelo final ao longo das épocas.

    Destaca numericamente os valores obtidos na última época.

    Parameters
    -----
    history : keras.callbacks.History
        Histórico retornado pelo método model.fit().
    """
    accuracy = history.history["accuracy"]
    loss = history.history["loss"]

    epocas = range(1, len(loss) + 1)

    ultima_epoca = len(loss)
    accuracy_final = accuracy[-1]
    loss_final = loss[-1]

    fig, axes = plt.subplots(
        nrows=1,
        ncols=2,
        figsize=(16, 5)
    )

    ax_accuracy, ax_loss = axes

    # =====
    # Acurácia
    # =====

    ax_accuracy.plot(
        epocas,
```

```

        accuracy,
        label="Treino"
    )

    ax_accuracy.scatter(
        ultima_epoca,
        accuracy_final,
        zorder=3
    )

    ax_accuracy.annotate(
        f"{accuracy_final:.4f}",
        xy=(ultima_epoca, accuracy_final),
        xytext=(-45, 15),
        textcoords="offset points",
        arrowprops={
            "arrowstyle": "->",
            "connectionstyle": "arc3"
        }
    )

    ax_accuracy.set_title(
        f"Evolução da acurácia do modelo final\n"
        f"Acurácia final: {accuracy_final:.4f}"
    )

    ax_accuracy.set_xlabel("Época")
    ax_accuracy.set_ylabel("Acurácia")
    ax_accuracy.set_xlim(1, ultima_epoca)
    ax_accuracy.grid(True, alpha=0.3)
    ax_accuracy.legend()

# =====
# Loss
# =====

    ax_loss.plot(
        epocas,
        loss,
        label="Treino"
    )

    ax_loss.scatter(
        ultima_epoca,
        loss_final,
        zorder=3
    )

```

```

ax_loss.annotate(
    f"{loss_final:.4f}",
    xy=(ultima_epoca, loss_final),
    xytext=(-45, 15),
    textcoords="offset points",
    arrowprops={
        "arrowstyle": "->",
        "connectionstyle": "arc3"
    }
)

ax_loss.set_title(
    f"Evolução da loss do modelo final\n"
    f"Loss final: {loss_final:.4f}"
)

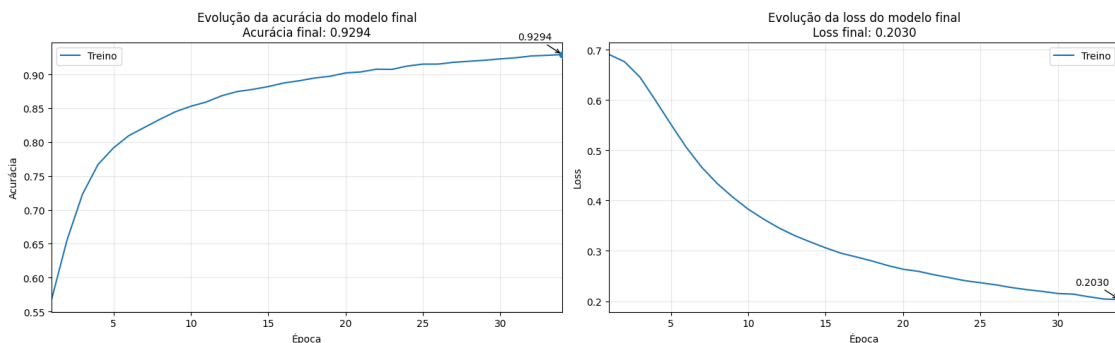
ax_loss.set_xlabel("Época")
ax_loss.set_ylabel("Loss")
ax_loss.set_xlim(1, ultima_epoca)
ax_loss.grid(True, alpha=0.3)
ax_loss.legend()

plt.tight_layout()
plt.show()

print("=== Métricas finais de treino ===")
print(f"Época final : {ultima_epoca}")
print(f"Acurácia      : {accuracy_final:.4f}")
print(f"Loss           : {loss_final:.4f}")

```

[62]: `plotar_historico_modelo_final(history_final)`



```

=== Métricas finais de treino ===
Época final : 34

```

Acurácia : 0.9294  
Loss : 0.2030

#### 1.1.11 6º Passo:

- Realize a avaliação do modelo com o método `.evaluate()`;
- **Imprima na tela 10** predições com seus respectivos valores verdadeiros.

```
[63]: # =====  
# Preparação do conjunto de teste  
# =====  
  
X_test_final = tf.convert_to_tensor(  
    X_test.astype(str).to_numpy().reshape(-1, 1),  
    dtype=tf.string  
)  
  
y_test_final = tf.convert_to_tensor(  
    y_test.to_numpy(),  
    dtype=tf.float32  
)  
  
print(f"Formato de X_test_final: {X_test_final.shape}")  
print(f"Formato de y_test_final: {y_test_final.shape}")
```

Formato de X\_test\_final: (5000, 1)  
Formato de y\_test\_final: (5000,)

```
[67]: # =====  
# Preparação do conjunto de teste  
# =====  
  
X_test_final = tf.convert_to_tensor(  
    X_test.astype(str).to_numpy().reshape(-1, 1),  
    dtype=tf.string  
)  
  
y_test_final = tf.convert_to_tensor(  
    y_test.to_numpy(),  
    dtype=tf.float32  
)  
  
print(f"Formato de X_test_final: {X_test_final.shape}")  
print(f"Formato de y_test_final: {y_test_final.shape}")  
  
# =====  
# Avaliação do modelo com .evaluate()  
# =====
```

```

test_loss, test_accuracy = model_final.evaluate(
    X_test_final,
    y_test_final,
    batch_size=FINAL_BATCH_SIZE,
    verbose=1
)

print("\n=== Avaliação final com model.evaluate() ===")
print(f"Loss      : {test_loss:.4f}")
print(f"Acurácia : {test_accuracy:.4f}")

# =====
# Predições sobre todo o conjunto de teste
# =====

y_prob = model_final.predict(
    X_test_final,
    batch_size=FINAL_BATCH_SIZE,
    verbose=1
).reshape(-1)

y_pred = (y_prob >= 0.5).astype(int)
y_true = y_test_final.numpy().astype(int)

# =====
# Métricas consolidadas
# =====

accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred)
recall = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)
roc_auc = roc_auc_score(y_true, y_prob)

print("\n=== Métricas consolidadas no conjunto de teste ===")
print(f"Acurácia   : {accuracy:.4f}")
print(f"Precisão    : {precision:.4f}")
print(f"Recall      : {recall:.4f}")
print(f"F1-score    : {f1:.4f}")
print(f"ROC-AUC     : {roc_auc:.4f}")

# =====
# Relatório detalhado por classe

```

```

# =====

print("\n=== Relatório de classificação ===")

print(
    classification_report(
        y_true,
        y_pred,
        target_names=["Negativa", "Positiva"],
        digits=4
    )
)

# =====
# Matriz de confusão
# =====

cm = confusion_matrix(y_true, y_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Negativa", "Positiva"]
)

fig, ax = plt.subplots(figsize=(6, 5))

disp.plot(
    ax=ax,
    values_format="d",
    colorbar=False
)

ax.set_title("Matriz de confusão do modelo final")

plt.tight_layout()
plt.show()

```

Formato de X\_test\_final: (5000, 1)

Formato de y\_test\_final: (5000,)

79/79 0s 6ms/step -

accuracy: 0.8994 - loss: 0.2499

=== Avaliação final com model.evaluate() ===

Loss : 0.2499

Acurácia : 0.8994

79/79 0s 4ms/step

=== Métricas consolidadas no conjunto de teste ===

Acurácia : 0.8994

Precisão : 0.8908

Recall : 0.9104

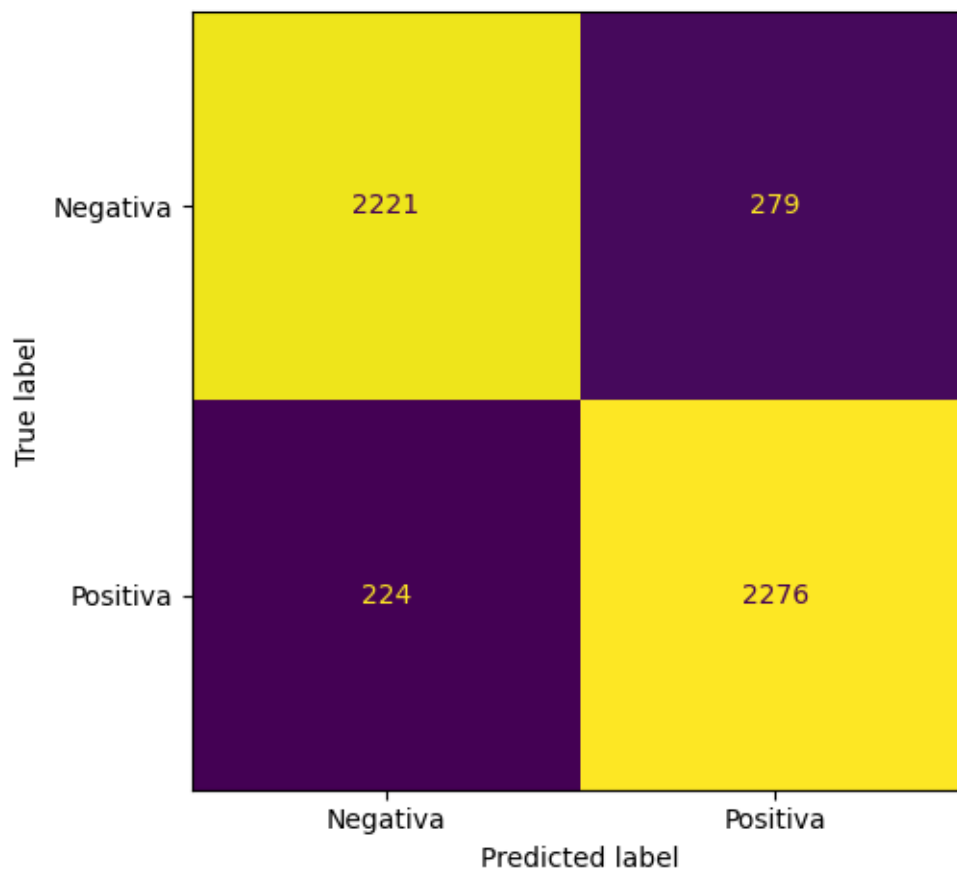
F1-score : 0.9005

ROC-AUC : 0.9629

=== Relatório de classificação ===

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Negativa     | 0.9084    | 0.8884 | 0.8983   | 2500    |
| Positiva     | 0.8908    | 0.9104 | 0.9005   | 2500    |
| accuracy     |           |        | 0.8994   | 5000    |
| macro avg    | 0.8996    | 0.8994 | 0.8994   | 5000    |
| weighted avg | 0.8996    | 0.8994 | 0.8994   | 5000    |

Matriz de confusão do modelo final



Executamos a avaliação final sobre o conjunto de teste reservado, composto por 5.000 reviews



balanceadas entre as classes positiva e negativa. O modelo obteve acurácia de 0,8994 e loss de 0,2499, mantendo desempenho próximo ao observado durante a etapa de validação.

A matriz de confusão mostra 2.221 reviews negativas e 2.276 reviews positivas classificadas corretamente. Observamos 279 falsos positivos e 224 falsos negativos. O modelo apresentou precisão de 0,8908, recall de 0,9104 e F1-score de 0,9005 para a classe positiva. Considerando ambas as classes, o F1-score macro foi de 0,8994.

O valor de ROC-AUC foi de 0,9629, indicando boa capacidade de separação entre reviews positivas e negativas. Os resultados sugerem que o modelo apresenta desempenho equilibrado entre as classes e capacidade adequada de generalização sobre dados não utilizados durante o treinamento.

A seguir exibimos alguns exemplos de classificações corretas e erradas em cada classe.

```
[70]: def selecionar_predicoes_para_inspecao(
    X_test,
    y_true,
    y_pred,
    y_prob,
    random_state=RANDOM_STATE
):
    """
    Seleciona 10 predições representativas para inspeção manual:

    - 2 verdadeiros positivos;
    - 2 verdadeiros negativos;
    - 3 falsos positivos;
    - 3 falsos negativos.

    Parameters
    -----
    X_test : pandas.Series
        Reviews originais do conjunto de teste.

    y_true : array-like
        Valores verdadeiros.

    y_pred : array-like
        Classes previstas após aplicação do limiar de decisão.

    y_prob : array-like
        Probabilidades estimadas para a classe positiva.

    random_state : int
        Semente utilizada para garantir reprodutibilidade.

    Returns
    -----
    pandas.DataFrame
```

```

    Tabela contendo as amostras selecionadas.
    """
    rng = np.random.default_rng(random_state)

    X_test_reset = X_test.reset_index(drop=True)

    y_true = np.asarray(y_true).astype(int)
    y_pred = np.asarray(y_pred).astype(int)
    y_prob = np.asarray(y_prob).reshape(-1)

    if not (
        len(X_test_reset)
        == len(y_true)
        == len(y_pred)
        == len(y_prob)
    ):
        raise ValueError(
            "X_test, y_true, y_pred e y_prob devem possuir "
            "a mesma quantidade de amostras."
        )

    categorias = {
        "Positiva correta": {
            "indices": np.where(
                (y_true == 1) & (y_pred == 1)
            )[0],
            "quantidade": 2
        },
        "Negativa correta": {
            "indices": np.where(
                (y_true == 0) & (y_pred == 0)
            )[0],
            "quantidade": 2
        },
        "Falso positivo": {
            "indices": np.where(
                (y_true == 0) & (y_pred == 1)
            )[0],
            "quantidade": 3
        },
        "Falso negativo": {
            "indices": np.where(
                (y_true == 1) & (y_pred == 0)
            )[0],
            "quantidade": 3
        }
    }
}

```

```
registros = []

for categoria, configuracao in categorias.items():
    indices_disponiveis = configuracao["indices"]
    quantidade = configuracao["quantidade"]

    if len(indices_disponiveis) < quantidade:
        raise ValueError(
            f"A categoria '{categoria}' possui apenas "
            f"{len(indices_disponiveis)} amostras disponíveis, "
            f"mas foram solicitadas {quantidade}."
        )

    indices_selecionados = rng.choice(
        indices_disponiveis,
        size=quantidade,
        replace=False
    )

    for indice in indices_selecionados:
        registros.append({
            "categoria": categoria,
            "índice no teste": indice,
            "valor verdadeiro": y_true[indice],
            "classe predita": y_pred[indice],
            "probabilidade positiva": y_prob[indice],
            "review": X_test_reset.iloc[indice]
        })

return pd.DataFrame(registros)
```

```
[72]: df_predicoes_inspecao = selecionar_predicoes_para_inspecao(
        X_test=X_test,
        y_true=y_true,
        y_pred=y_pred,
        y_prob=y_prob,
        random_state=RANDOM_STATE
    )

pd.set_option("display.max_colwidth", None)

df_predicoes_inspecao.style.format({
    "probabilidade positiva": "{:.4f}"
})
```

```
[72]: <pandas.io.formats.style.Styler at 0x34dcd2720>
```

A inspeção manual das predições mostra que o modelo identifica corretamente reviews com polaridade claramente definida. Textos com predominância de elogios recebem probabilidades elevadas para a classe positiva, enquanto críticas explícitas tendem a ser classificadas corretamente como negativas.

Os erros ocorrem principalmente em reviews ambíguas, que combinam elogios e críticas, ou em textos nos quais palavras negativas descrevem o enredo do filme sem representar uma avaliação desfavorável. Também foram observados casos próximos ao limiar de decisão de 0,5, indicando baixa confiança do modelo.

Esses resultados são compatíveis com a arquitetura utilizada. A camada `GlobalAveragePooling1D` produz uma representação compacta da review por meio da média dos embeddings, mas não considera diretamente a ordem das palavras, relações de negação ou contrastes entre trechos. Essa limitação pode dificultar a interpretação de textos longos ou semanticamente mais complexos.

#### 1.1.12 8º Passo:

- Faça o **relatório da modelagem da arquitetura neural** em uma célula *markdown*. Descreva alterações que você teve de realizar na arquitetura a fim de aprimorar a performance do predição.

Foi desenvolvida uma arquitetura neural para classificação binária de sentimentos em reviews do dataset IMDB. O modelo recebe diretamente uma review textual e produz uma probabilidade associada à classe positiva. Reviews com probabilidade maior ou igual a 0,5 são classificadas como positivas; as demais são classificadas como negativas.

**Pré-processamento e preparação dos textos** Antes da definição da arquitetura, foi realizada uma análise exploratória dos textos. O pré-processamento adotado converte os caracteres para minúsculas, remove tags HTML, elimina caracteres especiais, normaliza espaços consecutivos e remove espaços excedentes no início e no final das reviews.

A remoção explícita das tags HTML foi necessária porque marcações como `<br />` estavam sendo transformadas no token artificial `br`. Antes desse ajuste, esse token aparecia entre os termos mais frequentes do corpus e introduzia ruído no vocabulário.

A análise do comprimento das reviews mostrou uma distribuição assimétrica à direita. Para evitar que poucos textos excepcionalmente longos determinassem o tamanho de todas as entradas, foi escolhido o percentil 95 da distribuição como limite máximo:

```
MAX_LENGTH = 581
```

Dessa forma, 95% das reviews são representadas integralmente. Textos maiores são truncados e textos menores recebem preenchimento até atingir o tamanho fixado.

O tamanho do vocabulário também foi definido com base na cobertura acumulada das ocorrências de palavras. A configuração escolhida foi:

```
VOCAB_SIZE = 15.000
```

Esse valor cobre aproximadamente 95,44% das ocorrências do corpus, mantendo apenas 4,56% das palavras fora do vocabulário. A escolha representa um equilíbrio entre cobertura textual e custo computacional.

**Arquitetura utilizada** A arquitetura final foi composta pelas seguintes camadas:

Entrada textual

- TextVectorization
- Embedding
- GlobalAveragePooling1D
- Dense
- Dropout
- Dense com ativação sigmoid

A camada `TextVectorization` aplica o pré-processamento e converte cada review em uma sequência de 581 índices inteiros.

A camada `Embedding` transforma cada índice em um vetor numérico treinável. A dimensão adotada foi:

```
EMBED_DIM = 50
```

Com um vocabulário de 15.000 palavras, essa camada contém 750.000 parâmetros treináveis.

Após o embedding, foi utilizada a camada `GlobalAveragePooling1D`. Essa camada reduz a matriz de representação de cada review a um único vetor médio. A escolha evita o aumento expressivo da quantidade de parâmetros que ocorreria com uma camada `Flatten`, mantendo a arquitetura mais compacta.

A camada intermediária final possui:

```
DENSE_UNITS = 16
```

neurônios com ativação `ReLU`. Em seguida, foi aplicado:

```
DROPOUT_RATE = 0.5
```

para reduzir o sobreajuste observado durante os treinamentos experimentais.

A camada de saída possui um único neurônio com ativação `sigmoid`, adequada ao problema de classificação binária.

**Ajuste dos hiperparâmetros** A seleção dos hiperparâmetros foi realizada em duas etapas com `KerasTuner`.

Inicialmente, foi executada uma busca aleatória com diferentes valores para:

- dimensão do embedding;
- quantidade de neurônios da camada densa;
- taxa de dropout;
- otimizador;
- taxa de aprendizado.

Essa etapa mostrou que o otimizador `Adam` apresentou desempenho superior às alternativas avaliadas. Também indicou que a dimensão de embedding igual a 50 era suficiente, pois valores maiores não produziram ganhos relevantes e aumentavam significativamente a quantidade de parâmetros.

Na etapa seguinte, foi realizada uma busca exaustiva em um espaço refinado, mantendo fixos o otimizador `Adam` e a dimensão do embedding. Foram avaliadas todas as 36 combinações possíveis

entre:

```
dense_units    = [16, 32, 64]
dropout_rate   = [0.2, 0.3, 0.4, 0.5]
learning_rate  = [5e-5, 1e-4, 2e-4]
```

Os modelos com melhor acurácia de validação apresentaram resultados muito próximos. Por esse motivo, a escolha final não considerou apenas o maior valor pontual de acurácia, mas também a estabilidade das curvas de treino e validação, a simplicidade da arquitetura e os sinais de sobreajuste.

Foi selecionada a configuração:

```
EMBED_DIM = 50
DENSE_UNITS = 16
DROPOUT_RATE = 0.5
LEARNING_RATE = 5e-5
OPTIMIZER = "adam"
```

Essa configuração apresentou convergência mais gradual e menor diferença entre as curvas de treino e validação. Além disso, utiliza uma camada densa menor e maior regularização por dropout, reduzindo a complexidade sem perda relevante de desempenho.

**Treinamento final e avaliação** Após a seleção da arquitetura, o modelo foi treinado novamente utilizando todas as 45.000 reviews disponíveis no conjunto de treino. O conjunto de teste permaneceu isolado durante as etapas de seleção e ajuste dos hiperparâmetros.

Na avaliação final sobre 5.000 reviews de teste, o modelo obteve:

```
Loss      = 0,2499
Acurácia  = 0,8994
Precisão  = 0,8908
Recall    = 0,9104
F1-score  = 0,9005
ROC-AUC   = 0,9629
```

A matriz de confusão apresentou:

```
2.221 reviews negativas classificadas corretamente
2.276 reviews positivas classificadas corretamente
  279 falsos positivos
  224 falsos negativos
```

Os resultados indicam desempenho equilibrado entre as classes e capacidade adequada de generalização sobre dados não utilizados durante o treinamento.